

OpenMind: Query-Time Diversification via Logical Rules

Wenfei Fan^{1,2,3}, Kehan Pang³, Dandan Lin¹

¹Shenzhen Institute of Computing Sciences ²University of Edinburgh ³Beihang University
{fanwenfei, lindandan}@sics.ac.cn, pangkehan@buaa.edu.cn

ABSTRACT

Recommender systems are a major driver of filter bubbles, in which users are repeatedly exposed to opinions aligned with their existing views. We propose OpenMind, a lightweight, query-time framework that extends recommendation queries with diverse, alternative and even opposing viewpoints. Rather than replacing or retraining existing recommendation models, OpenMind augments query results by returning complementary items as optional extended answers, preserving both efficiency and accuracy. At the core of OpenMind is a class of logical rules, called OMRs (Open Mind Rules), which operate over a user-item interaction graph to identify high-quality content that differs in perspective while remaining relevant and engaging. We develop efficient algorithms that integrate OMRs into both click-through-rate (CTR) and top- k query processing, executing in the same inference pass as the underlying recommender. Using real-world datasets, we empirically verify that OpenMind improves exposure to diverse views at negligible query processing cost without loss in accuracy.

PVLDB Artifact Availability: The source code, data, and/or other artifacts have been made available at <https://github.com/hdqlkncso24jvf/OpenMind>.

1 INTRODUCTION

Frank and Nancy watched a short video of a debate between men and women. Although the two watched the same clip, the video platform served them entirely different comments. Frank only saw comments supporting the male perspective, while Nancy was presented with comments criticizing men as unreasonable. This disparity ultimately led to a heated argument between them.

This example illustrates how recommender systems shape the information we encounter online. By continuously adapting recommendations to users’ historical interactions, modern recommendation algorithms maximize engagement by favoring content aligned with users’ prior views. As a result, users are repeatedly exposed to opinions they already agree with, while alternative perspectives are rarely surfaced. This phenomenon is known as a *filter bubble* (*a.k.a.* echo chamber, information cocoon) [33, 63, 65].

Breaking free from filter bubbles is critical: they hinder creativity and innovation [60, 62], undermine democratic processes [21, 77], and increase vulnerability to manipulation and misinformation. Empirical study shows that falsehoods spread “farther, faster, deeper, and more broadly” than truths, with homophilous groups propagating false news 2–3× faster than mixed groups [83]. At the same time, users seek diverse opinions to understand the spectrum of views, validate beliefs and construct counterarguments [10, 34].

Despite its importance, existing approaches to breaking filter bubbles fall into two main categories and remain limited.

Diverse recommendation via re-ranking. A host of work promotes diversity by re-ranking candidate items returned by a base recommender (*e.g.*, [13, 15, 22, 23, 57, 64, 67, 88]). These methods typically

target *topic-level* diversity, ensuring that recommendation lists span multiple subject areas. However, filter bubbles often arise from exposure to only one *perspective* on a topic. For instance, although both Frank and Nancy watched the same debate video, the platform showed each of them comments aligned with their respective views. Moreover, re-ranking operates on pre-retrieved candidates that are already *biased* toward the user’s historical preferences.

Breaking filter bubbles via influence propagation. Another line of work studies filter bubbles in social networks using influence maximization techniques (*e.g.*, [11, 31, 32, 41, 61, 79]). Given the placement of an item a , these methods identify k users where an opposing item b should be injected to maximize its propagation. While effective for analyzing the *global diffusion* of opinions, they do not address how recommender systems should expose *individual users* to alternative viewpoints during recommendation query answering.

Consequently, it remains unclear how recommender systems can suggest alternative viewpoints for individual users during query answering, without retraining models or degrading performance.

This issue is challenging due to several requirements.

(1) Diversity. Given a recommender $\mathcal{A}$ that suggests items $I(x)$ for user x , the system should generate a complementary set $I_d(x)$ offering alternative or opposing views on the same topic while remaining relevant and engaging. It should also justify the suggestions.

(2) Lightweight integration. Since modern recommenders are expensive to train and operate under strict latency constraints [42, 54, 89, 91], the solution should not require retraining and must integrate with $\mathcal{A}$ during inference, identifying $I_d(x)$ in the same pass used to compute $I(x)$ without degrading throughput or accuracy.

(3) Model generality. The approach should apply to different recommendation settings, including (a) *CTR-based recommenders*, which rank items by predicted click-through-rate probability, and (b) *top- k recommenders*, which return the k highest-scoring items.

OpenMind. To address these challenges, we propose OpenMind, a lightweight framework that extends existing recommenders with *viewpoint-level* diversification. Unlike prior approaches that promote topic diversity, OpenMind identifies *alternative perspectives on the same topic*. Given a recommender that produces a set $I(x)$ of items for a user x , OpenMind finds a complementary set $I_d(x)$ of items that present different or opposing viewpoints while remaining relevant and engaging. These alternatives are suggested as optional extended readings, enabling users to explore diverse perspectives without disrupting the original recommendation process.

The key idea is a class of *Open Mind Rules* (OMRs) that combine logical reasoning with machine-learning (ML). Each rule is defined over a graph pattern and a dependency: the pattern captures the recommendation context by linking a user, an anchor item, and candidate alternatives; while the dependency specifies when an alternative should be suggested. The rule conditions integrate sig-

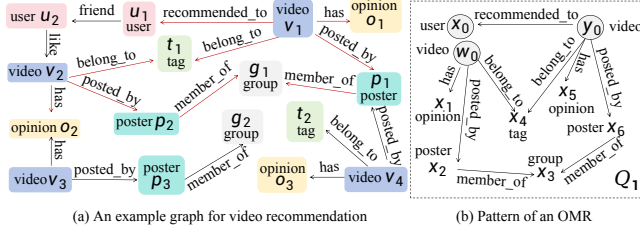


Figure 1: A graph and a pattern for video recommendation

nals such as recommendation scores, content dissimilarity and item popularity, such that alternatives are both diverse and acceptable.

The novelty of OpenMind lies in its *lightweight*, *efficient*, and *general* design. It augments arbitrary recommenders, whether ML- or logic-based, without retraining the model. Moreover, anchor items $I(x)$ and alternative viewpoints $I_d(x)$ are derived in the *same inference pass* by tightly coupling their evaluation within the recommender, rather than in separate stages. Finally, OpenMind applies to both major settings, *i.e.*, CTR-based and top- k recommenders.

Contributions & Organization. Our contributions are as follows.

(1) *Logic-based framework* (Section 2). We introduce OMRs (Open Mind Rules), which incorporate ML predicates, including the recommendation model M_{rec} of a recommender $\mathcal{A}$ and auxiliary signals for diversity and popularity. Given an item y recommended to user x , OMRs identify alternative items w that are diverse yet acceptable.

(2) *Model-agnostic integration* (Section 3). We propose OpenMind, a plug-and-play framework that augments arbitrary recommenders $\mathcal{A}$ with alternative views, without retraining M_{rec} . It discovers a set Σ of OMRs from a user-item interaction graph and applies them to find complementary items $I_d(x)$ together with the anchor set $I(x)$.

(3) *Efficient recommendation algorithms* (Sections 4 and 5). We develop two algorithms, CTR and TopK, for CTR-based and top- k settings. They couple anchor inference with alternative generation so both are computed in a single inference pass, avoiding the sequential pipeline of first identifying anchors and then deriving alternatives. This enables shared refinement and early pruning. We prove the coupled execution sound and complete *w.r.t.* the sequential baseline, returning the same results with improved efficiency.

(4) *Empirical results* (Section 6). Using real-world datasets, we find the following. (a) *Effectiveness*. For CTR, while retaining comparable accuracy, OpenMind improves the diversity metrics ILD and FBP of SOTA (state-of-the-art) methods by 35.62% and 46.06% on average, respectively, up to 110.28% and 144.30%. Top- k improves diversity ILD@ k and FBP@ k by 42.19% and 45.98%, respectively. (b) *Efficiency*. Answering a CTR query for a user takes 0.12s on average on graphs with 225M edges, 13.7× faster than the sequential pipeline. For top- k , it takes 86ms on average, up to 16.4× faster. (c) *Optimization*. Our pruning strategies further speed up by up to 1.89×.

Related and future work are in Sections 7 and 8; proofs are in [4].

2 RULES FOR KEEPING AN OPEN MIND

This section presents OMRs, Open Mind Rules. We first review basic notations (Section 2.1), and then introduce OMRs (Section 2.2).

2.1 Preliminaries

We start with notations. Assume a countably infinite set Λ of labels.

Graphs. Consider directed labeled graphs $G = (V, E, L, F_A)$, where

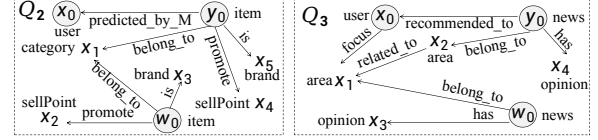


Figure 2: Graph patterns

(a) V is a finite set of vertices; (b) $E \subseteq V \times \Lambda \times V$ is a finite set of labeled edges, where (v, l, v') denotes an edge labeled $l \in \Lambda$ from v to v' ; and (c) each vertex $v \in V$ has a label $L(v) \in \Lambda$ and a finite tuple of attributes $F_A(v) = (A_1 = a_1, \dots, A_n = a_n)$, written as $v.A_i = a_i$. Vertices may have different attributes without a fixed schema.

Patterns. A graph pattern is $Q[\bar{x}, x_0, y_0, w_0] = (V_Q, E_Q, L_Q)$, where (1) V_Q (resp. E_Q) is a finite set of *pattern vertices* (resp. *edges*); (2) L_Q assigns each pattern vertex $v \in V_Q$ a label $L_Q(v) \in \Lambda$; (3) $\bar{x}$ is a list of distinct variables, each denoting a vertex v in V_Q ; and (4) $x_0 \in \bar{x}$ denotes a user, and $y_0 \in \bar{x}$ and $w_0 \in \bar{x}$ denote two items on the same topic but expressing different viewpoints. We call y_0 (resp. w_0) the *anchor* (resp. *alternative*) item variable. We use $Q[x_0, y_0, w_0]$ and $Q[\bar{x}, x_0, y_0, w_0]$ interchangeably when clear from context.

Pattern matching. A *match* of pattern $Q[\bar{x}, x_0, y_0, w_0]$ in graph G is defined as a homomorphism h from Q to G such that (a) for each pattern vertex $v \in V_Q$, $L_Q(v) = L(h(v))$; and (b) for each pattern edge $e = (v, l, v')$ in Q , $e' = (h(v), l, h(v'))$ is an edge in G . We denote the match by the vector $h(\bar{x})$, consisting of $h(x)$ for all $x \in \bar{x}$.

Example 1: Figure 1(a) shows a labeled graph G for video recommendation. Figure 1(b) depicts a graph pattern $Q_1[\bar{x}, x_0, y_0, w_0]$ that explains why video w_0 is recommended to user x_0 as an alternative to anchor y_0 . A match of Q_1 in Figure 1(a) is highlighted in red. $\square$

2.2 Open Mind Rules

We next define OMRs, starting with their predicates.

Predicates. A *predicate* over pattern $Q[\bar{x}, x_0, y_0, w_0]$ has the form:

$$x.A \oplus y.B \mid x.A \oplus c \mid \mathcal{M}(x, y) \otimes \delta \mid \mathcal{M}(y) \otimes \delta \mid \text{Rec}(x, y),$$

where $\oplus$ is one of $=, \neq, <, \leq, >, \geq$; $\otimes$ is one of $<, \leq, >, \geq$; x and y are variables in $\bar{x}$; c is a constant; A and B are attributes in Υ ; $\text{Rec}(x, y)$ is a predicate that is true when item y is recommended to user x , and $\mathcal{M}(x, y)$ is an ML predicate, including:

- A recommendation model $M_{rec}(x, y)$ outputs a score in $[0, 1]$ indicating user preference. It may be (a) CF-based (e.g., NMF [59], NCF [39], LightGCN [37]), (b) CB-based (e.g., LightGBM [46], LSTUR [8]), or (c) hybrid (e.g., DeepFM [35], xDeepFM [51]).
- A model $M_{dis}(y, w)$ that measures item dissimilarity via classical distance metrics (e.g., cosine similarity [50, 68], mean squared difference [9, 71]), random walk-based metrics (e.g., random walk with restart [55], SimRank [45]), or metric-learning methods (e.g., CML [40], SML [90], SimCLR [16], SelfDiss [92]).
- A model $M_{agg}(y)$ that estimates item popularity using click/rating/exposure frequency [93, 96], power-law modeling [20], regression-based methods [7, 70], or RL-based approaches [72].

All prediction scores produced by $\mathcal{M}$ are normalized to $[0, 1]$ and compared against a predefined threshold $\delta \in [0, 1]$.

OMRs. An OMR (Open Mind Rule) ϕ has the form:

$$Q[x_0, y_0, w_0](X \rightarrow \text{Rec}_d(x_0, w_0)),$$

where $Q[x_0, y_0, w_0]$ is a graph pattern; $\text{Rec}_d(x_0, w_0)$ is true when item w_0 is recommended to user x_0 as an alternative to y_0 ; and X is

a conjunction of predicates over $Q[x_0, y_0, w_0]$ that include at least one predicate of either the form $\text{Rec}(x_0, y_0)$ or $\mathcal{M}_{\text{rec}}(x_0, y_0) \otimes \delta$.

We refer to Q and $X \rightarrow \text{Rec}_d(x_0, w_0)$ as the *pattern* and *dependency*, and X and $\text{Rec}_d(x_0, w_0)$ as the *precondition* and *consequence* of φ , respectively. We consider OMRs that pertain to the same recommendation model $\mathcal{M}_{\text{rec}}$, referred to as OMRs *pertaining to* $\mathcal{M}_{\text{rec}}$.

Intuitively, φ recommends alternative w_0 to user x_0 by referencing an anchor y_0 already suggested to x_0 by $\text{Rec}(x_0, y_0)$ or $\mathcal{M}_{\text{rec}}(x_0, y_0) \otimes \delta$. We separate the pattern Q from the condition X to (a) visualize the topology of the entities via Q , and (b) speed up evaluation by exploiting the locality of pattern matching. Here Rec may appear in the precondition X , not only in the consequence of φ .

Example 2: Below are OMRs with patterns in Figures 1(b) and 2.

(1) An OMR φ_1 is $Q_1[x_0, y_0, w_0](\mathcal{M}_{\text{rec}}(x_0, y_0) \wedge X_1 \rightarrow \text{Rec}_d(x_0, w_0))$, where X_1 is $\mathcal{M}_{\text{dis}}(y_0, w_0) \geq \delta_1 \wedge \mathcal{M}_{\text{aggr}}(w_0) \geq \delta_2 \wedge x_2.\text{followers} \geq 1000 \wedge x_1.\text{type} = \text{pro} \wedge x_5.\text{type} = \text{anti}$. The rule suggests video w_0 to user x_0 when (a) recommender $\mathcal{M}_{\text{rec}}$ already suggests video y_0 to x_0 ; (b) w_0 concerns the same topic as y_0 but is sufficiently different (measured by $\mathcal{M}_{\text{dis}}$); (c) w_0 is sufficiently popular (by $\mathcal{M}_{\text{aggr}}$); and (d) w_0 is posted by a creator x_2 with at least 1000 followers and belongs to the same group x_3 as the poster x_6 of y_0 . Thus, φ_1 exposes user x_0 to qualified content from the opposite viewpoint, while maintaining a high likelihood of engagement by selecting w_0 from the same community as the poster of y_0 .

(2) $\varphi_2 = Q_2[x_0, y_0, w_0](\mathcal{M}_{\text{rec}}(x_0, y_0) \geq \delta \wedge X_2 \rightarrow \text{Rec}_d(x_0, w_0))$, where X_2 is $\mathcal{M}_{\text{aggr}}(w_0) \geq \delta_1 \wedge w_0.\text{rate} \geq 8.0 \wedge x_3.\text{id} \neq x_5.\text{id} \wedge x_2.\text{name} = \text{cheap} \wedge x_4.\text{name} = \text{highly-effective}$. It suggests item w_0 to user x_0 when (a) $\mathcal{M}_{\text{rec}}$ already recommends item y_0 to x_0 ; (b) w_0 and y_0 are in the same category x_1 ; (c) w_0 is popular (by $\mathcal{M}_{\text{aggr}}$) and is highly rated (average rating > 8.0); (d) w_0 and y_0 have different brands; and (e) w_0 emphasizes a “cheap” selling point, whereas y_0 emphasizes “high-effectiveness”. This exposes user x_0 to items with diverse brands and selling points, beyond their prior interactions.

(3) $\varphi_3 = Q_3[x_0, y_0, w_0](\text{Rec}(x_0, y_0) \wedge X_3 \rightarrow \text{Rec}_d(x_0, w_0))$, where X_3 is $x_1.\text{name} = \text{DB} \wedge x_2.\text{name} = \text{AI} \wedge \mathcal{M}_{\text{aggr}}(w_0) \geq \delta_1 \wedge y_0.\text{topic} = w_0.\text{topic} \wedge x_3.\text{type} = \text{Yes} \wedge x_5.\text{type} = \text{No}$. It suggests news w_0 to user x_0 when (a) news y_0 is already recommended to x_0 , (b) y_0 is AI-related and connected to the DB topic x_1 that x_0 follows; (c) a DB-related news w_0 is popular according to $\mathcal{M}_{\text{aggr}}$; and (d) w_0 and y_0 discuss the same topic but express different opinions. This rule broadens user x_0 ’s perspective by introducing a news item w_0 from a different angle while remaining within the same topic. $\square$

Semantics. Denote by h a match of the pattern Q of an OMR φ in graph G , and by p a predicate in X . We say that h *satisfies* p , written $h \models p$, if (a) If p is $x.A \oplus y.B$, then attributes A and B exist at vertices $h(x)$ and $h(y)$, respectively, and $h(x).A \oplus h(y).B$ holds; similarly for $x.A \oplus c$. (b) If p is $\mathcal{M}_{\text{rec}}(x, y) \otimes \delta$, then the ML model $\mathcal{M}_{\text{rec}}$ predicts that the strength of user x liking item y satisfies the threshold condition $\otimes \delta$; likewise for $\mathcal{M}_{\text{dis}}(y, w) \otimes \delta$ and $\mathcal{M}_{\text{aggr}}(w) \otimes \delta$. (c) If p is $\text{Rec}(x, y)$, then there exists a (possibly hidden) edge labeled Rec from $h(x)$ to $h(y)$, meaning item y is recommended to user x .

For a set X of predicates, $h \models X$ means $h \models p$ for every $p \in X$. For an OMR $\varphi = Q[x_0, y_0, w_0](X \rightarrow \text{Rec}_d(x_0, w_0))$ and a graph G , we write $G \models \varphi$ if every match h of Q in G that satisfies X also satisfies $\text{Rec}_d(x_0, w_0)$; i.e., item w_0 should be recommended to user x_0 .

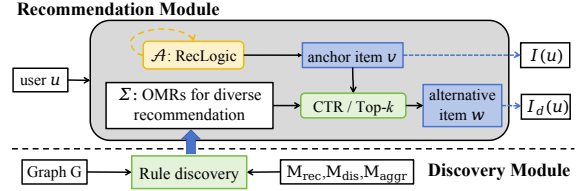


Figure 3: Recommendation framework OpenMind

3 MAKING RECOMMENDERS OPEN-MINDED

This section presents OpenMind, our recommendation framework.

Overview. As illustrated in Figure 3, OpenMind extends existing recommender systems $\mathcal{A}$ by explicitly incorporating diverse viewpoints. It operates as a lightweight, model-agnostic layer on top of any traditional $\mathcal{A}$ for news, videos, comments, articles, books and similar content. Given a user-item interaction graph G (optionally augmented with content features or knowledge-graph information), OpenMind first discovers a set Σ of OMRs from G . At recommendation time, for each user u , when $\mathcal{A}$ produces a set $I(u)$ of items predicted to match the user’s preferences, OpenMind applies the rules in Σ to generate a complementary set $I_d(u)$ of items that present alternative or opposing viewpoints to those in $I(u)$.

Both anchor set $I(u)$ and alternative set $I_d(u)$ are shown to user u , with alternatives explicitly labeled and presented alongside anchors, for voluntary exploration without disrupting the underlying recommender. Moreover, OpenMind can adapt existing acceptance and propagation metrics [11, 31, 52, 53] to selectively offer alternative items only to users who are likely to engage with them.

Implementation. In principle, OpenMind can be integrated with any recommender model $\mathcal{M}_{\text{rec}}$, as it augments $\mathcal{M}_{\text{rec}}$ with additional logical conditions to suggest alternative viewpoints.

We implement OpenMind on RecLogic [27], for three reasons. (1) Both are logic-based, enabling unified discovery of OMRs and TIEs. (2) Generating alternatives over RecLogic’s outputs is more challenging than over raw ML $\mathcal{M}_{\text{rec}}$, as logical constraints are already enforced during inference. (3) RecLogic improves ML accuracy by 22.89% on average when model confidence is intermediate [27].

RecLogic. Below we outline how RecLogic works. Given a recommendation model $\mathcal{M}_{\text{rec}}$, RecLogic aims to reduce its false positives (FPs) and false negatives (FNs). To achieve this, RecLogic employs a class of (predicTion rule)s, denoted TIEs, defined as follows.

(a) $Q[x_0, y_0](\mathcal{M}_{\text{rec}}(x_0, y_0) < \delta \wedge X_1 \rightarrow \text{Rec}(x_0, y_0))$ with logical precondition X_1 defined in terms of predicates $z_1.A \oplus z_2.B$ and $z.A \oplus c$. That is, although $\mathcal{M}_{\text{rec}}(x_0, y_0)$ predicts that user x_0 may not like item y_0 (i.e., $\mathcal{M}_{\text{rec}}(x_0, y_0) < \delta$), if conditions in X_1 hold, then RecLogic overrides the prediction of $\mathcal{M}_{\text{rec}}$ and recommends y_0 to x_0 . This reduces possible FNs of ML predictions $\mathcal{M}_{\text{rec}}(x_0, y_0)$.

(b) $Q[x_0, y_0](\mathcal{M}_{\text{rec}}(x_0, y_0) \geq \delta \wedge X_1 \rightarrow \text{Rec}(x_0, y_0))$. That is, while $\mathcal{M}_{\text{rec}}(x_0, y_0)$ suggests recommending item y_0 to user x_0 (i.e., $\mathcal{M}_{\text{rec}}(x_0, y_0) \geq \delta$), RecLogic uses additional conditions in X_1 to filter FPs, i.e., it does not recommend y_0 to x_0 if X_1 is not satisfied.

(c) $Q[x_0, y_0](\mathcal{M}_{\text{rec}}(x_0, y_0) \geq \delta_L \wedge \mathcal{M}_{\text{rec}}(x_0, y_0) \leq \delta_H \wedge X_1 \rightarrow \text{Rec}(x_0, y_0))$. If the strength of $\mathcal{M}_{\text{rec}}(x_0, y_0)$ is in a “fuzzy area” $[\delta_L, \delta_H]$ where the prediction strength is neither sufficiently high nor sufficiently low, RecLogic makes suggestion based on condition

X_1 rather than on $\mathcal{M}_{\text{rec}}$, to reduce FPs and FNs of ML predictions.

On top of RecLogic, OpenMind supports modules for rule discovery, CTR recommendations and top- k recommendations.

Rule discovery. Given a graph as described earlier, OpenMind discovers a set Σ of OMRs by building on the TIE discovery algorithm in RecLogic [27]. Specifically, (1) it augments G with (dummy) edges Rec , $\mathcal{M}_{\text{rec}}$, $\mathcal{M}_{\text{dis}}$ and attribute $\mathcal{M}_{\text{aggr}}$, to incorporate prediction signals into pattern matching. (2) It generalizes star-shaped pattern mining to generic patterns. (3) It learns OMRs jointly with TIEs to capture OMR-specific conditions. Details are deferred to [4].

Model $\mathcal{M}_{\text{dis}}(z_1, z_2)$. Given a user-item interaction graph G and two items z_1 and z_2 from the same category, we compute their dissimilarity score $\mathcal{M}_{\text{dis}}(z_1, z_2)$ using Random Walk with Restart (RWR) [55].

The key idea is to measure how the local neighborhoods of z_1 and z_2 split into opposing communities under RWR. We first induce an r -hop subgraph $SG(z_1, z_2)$ around the two items and partition it into two communities. For each $z_i \in \{z_1, z_2\}$, we select the top- r associated *authority vertices* as seeds S_{z_i} and run RWR on $SG(z_1, z_2)$. We then compute the probability that a walk starting from S_{z_i} remains in the same community as z_i . The dissimilarity score $\mathcal{M}_{\text{dis}}(z_1, z_2)$ is the difference between the probability of staying within the originating community and that of crossing to the other community. Larger values indicate greater dissimilarity. For items from different categories, we set $\mathcal{M}_{\text{dis}}(z_1, z_2) = 1$.

Model $\mathcal{M}_{\text{aggr}}(z)$. We define the popularity of an item z as the frequency of z that has been interacted by users, e.g., clicked, rated.

Recommendation settings. OpenMind supports two recommendation modes, CTR (click-through-rate) and top- k , each with a dedicated algorithm. A naive sequential solution first computes anchor items $I(x_0)$ using RecLogic and then enumerates alternative items $I_d(x_0)$ by applying OMRs, which is inefficient.

To address this, OpenMind tightly couples anchor selection and alternative generation within a single inference process. CTR jointly identifies anchor items and their most promising alternatives during inference, while TopK further incorporates ranking and pruning to efficiently return only the highest-scoring anchor-alternative pairs. We develop CTR and TopK in Sections 4 and 5, respectively.

Example 3: Consider the video clip mentioned in Section 1, where Frank and Nancy watched a debate on gender roles. Based on Frank’s past preference, CTR selects a set of highly ranked comments as *anchors*, each aligned with his own views. In the same inference pass, CTR recommends for each anchor a corresponding *alternative* comment that offers a contrasting comment of the clip, based on sociological evidence rather than personal opinion. Each anchor-alternative pair is topically aligned and widely discussed, but presents clearly different or even opposing viewpoints.

Assume $k = 2$. TopK selects two anchor comments for Frank with the highest recommendation strengths under RecLogic. One supports traditional family roles, while the other argues for their social value. For the two anchors, TopK recommends two alternatives: one questions the empirical basis of such roles, while the other offers a different cultural interpretation. All recommendations are tied to the same video, illustrating how OpenMind introduces viewpoint diversity without disrupting the underlying recommender. $\square$

OpenMind vs. RecLogic [27]. The two systems differ in their objectives, rule classes and algorithms. (1) *Objective.* OpenMind augments a recommender $\mathcal{A}$ with alternative viewpoints, whereas RecLogic augments an ML model $\mathcal{M}_{\text{rec}}$ to reduce FPs and FNs. (2) *Rules.* OpenMind uses OMRs, while RecLogic uses TIEs, differing in graph patterns and ML predicates. (3) *Algorithms.* RecLogic supports only CTR, whereas OpenMind supports both CTR and TopK.

OMRs subsume prediction rules (TIEs) of [27] as a special case. (1) OMRs aim to recommend items with diverse viewpoints to users, whereas TIEs are designed to reduce the FPs and FNs of $\mathcal{M}_{\text{rec}}$ [27]. That said, OMRs can be adapted to reduce FPs/FNs of $\mathcal{M}_{\text{rec}}$ via rules of the form $Q[x_0, y_0, w_0](\mathcal{M}_{\text{rec}}(x_0, y_0) \wedge X_1 \rightarrow \text{Rec}(x_0, y_0))$, which enrich $\mathcal{M}_{\text{rec}}$ ’s predictions with additional logical conditions X_1 . (2) OMRs are defined using general graph patterns, whereas TIEs restrict rules to star-shaped dual patterns. Moreover, (3) unlike TIEs that enrich $\mathcal{M}_{\text{rec}}(x_0, y_0)$ only, OMRs allow multiple heterogeneous ML predicates within a single rule, including binary predicates $\mathcal{M}_{\text{rec}}(x_0, y_0)$ and $\mathcal{M}_{\text{dis}}(y_0, w_0)$, and unary predicates $\mathcal{M}_{\text{aggr}}(w_0)$.

4 CTR RECOMMENDATION

This section develops CTR, the algorithm of OpenMind for click-through-rate recommendations enriched with diverse viewpoints. We formalize the CTR recommendation problem as follows.

- **Input:** A user-item interaction graph G , recommender RecLogic with a set Σ_{TIE} of TIEs, a user u and a set Σ_{OMR} of OMRs
- **Output:** A set $I(u)$ of anchors by RecLogic and a set $I_d(u)$ of highest-scoring alternatives by Σ_{OMR} such that for each item $w \in I_d(u)$, there exists at least one match h of an OMR ϕ in G such that $h \models \text{Rec}_d(u, w)$ by referencing an anchor $v \in I(u)$.

Challenges. The brute-force sequential solution outlined in Section 3 is expensive, as even checking the existence of matches for a general pattern Q in an OMR is NP-hard [30]. Moreover, for a given user u and anchor item v , multiple matches of a rule $\phi \in \Sigma_{\text{OMR}}$ may exist. Evaluating all such candidates is inefficient, and many may be weakly relevant or insufficiently distinct from v .

In light of these, we propose an *efficient score-based* CTR algorithm. For efficiency, we propose a *tightly coupled recommendation mechanism* where TIEs and OMRs *jointly* refine a shared candidate space through a single fixpoint computation. To avoid the enumeration of all matches by OMRs in Σ_{OMR} , we define a *fine-grained ranking score* over alternative items that jointly accounts for content dissimilarity (relative to the anchor items) and acceptability of the user, to recommend diverse yet favorable items. We define ranking scores in Section 4.1 and present algorithm CTR in Section 4.2.

4.1 Ranking Score

Given a user-anchor pair (u, v) , we aim to select an alternative item w that presents a different viewpoint from v without reducing user satisfaction. Emphasizing dissimilarity alone may lead to churn, while focusing solely on user satisfaction fails to provide sufficient diversity. To balance these objectives, we define a ranking score $s(u, v, w)$ that jointly considers dissimilarity and acceptability:

$$s(u, v, w) = \lambda \cdot s_{\text{dis}}(u, v, w) + (1 - \lambda) \cdot s_{\text{acc}}(u, w),$$

where $s_{\text{dis}}(u, v, w)$ measures viewpoint dissimilarity, $s_{\text{acc}}(u, w)$ captures user acceptability, and $\lambda \in [0, 1]$ controls the tradeoff. A higher score indicates a more promising alternative.

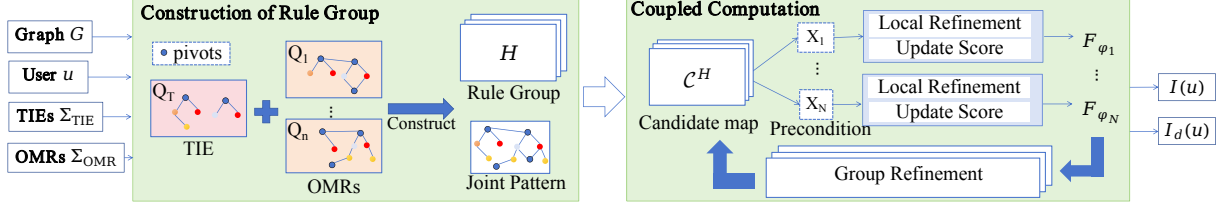


Figure 4: Overview of coupled CTR algorithm.

Next, we elaborate the dissimilarity score and acceptability score.

Dissimilarity score. The dissimilarity score $s_{\text{dis}}(u, v, w)$ quantifies how well item w serves as an alternative to anchor v for user u . An effective alternative should remain topically related to v while presenting sufficiently different content. Thus we define $s_{\text{dis}}(u, v, w)$ by combining topic proximity and content dissimilarity. Since different OMRs capture distinct structural and feature-based signals, their individual contributions are also incorporated into the score:

$$s_{\text{dis}}(u, v, w) = \frac{1}{|\Sigma_{\text{OMR}}|} \cdot \text{sim}_T(v, w) \cdot \sum_{\varphi \in \Sigma_{\text{OMR}}^w} s_{\text{dis}}(u, v, w, \varphi),$$

where $\text{sim}_T(v, w)$ is the topic proximity of item w w.r.t. item v , Σ_{OMR}^w is the set of OMRs that have at least one match h in G at (u, v) such that $h(w_0) = w$, and $s_{\text{dis}}(u, v, w, \varphi) \in [0, 1]$ is the individual content dissimilarity of w w.r.t. v under an OMR φ in Σ_{OMR}^w .

The individual dissimilarity $s_{\text{dis}}(u, v, w, \varphi)$ under each individual OMR $\varphi \in \Sigma_{\text{OMR}}^w$ is formally defined as follows:

$$s_{\text{dis}}(u, v, w, \varphi) = \mathcal{M}_{\text{dis}}(v, w) \cdot \Delta_{\varphi}(u, v, w),$$

where $\Delta_{\varphi}(u, v, w)$ is the contribution of dissimilarity given by φ in terms of topology and vertex features (see more details in [4]).

Let $\gamma(v, w) = \text{sim}_T(v, w) \cdot \mathcal{M}_{\text{dis}}(v, w)$. Then, given a set Σ_{OMR} of OMRs, the content dissimilarity score of w becomes:

$$s_{\text{dis}}(u, v, w, \Sigma_{\text{OMR}}) = \frac{1}{|\Sigma_{\text{OMR}}|} \cdot \gamma(v, w) \cdot \sum_{\varphi \in \Sigma_{\text{OMR}}^w} \Delta_{\varphi}(u, v, w).$$

Acceptability score. To mitigate churn from opposing viewpoints, we estimate a user’s likelihood of engaging with an alternative item. Let $\mathcal{L}(u, v, \Sigma_{\text{OMR}})$ denote the alternatives at (u, v) generated by OMRs in Σ_{OMR} . For each $w \in \mathcal{L}(u, v, \Sigma_{\text{OMR}})$, we define

$$s_{\text{acc}}(u, w) = \mathcal{M}(u, w) \cdot \mathcal{M}_{\text{aggr}}(w),$$

where $\mathcal{M}(u, w) \in [0, 1]$ is the predicted probability that user u engages with w , and $\mathcal{M}_{\text{aggr}}(w)$ measures its global popularity. Intuitively, w is more acceptable if it aligns with user u ’s interests and is broadly endorsed, reducing abrupt exposure to opposing views.

Score-based CTR problem. Based on the ranking score above, we reformulate the CTR task as follows. For each user u , RecLogic identifies a set of anchors $I(u)$. For each $v \in I(u)$, multiple candidate alternatives can be derived via OMRs, and the objective is to select the item $w \in \mathcal{L}(u, v, \Sigma_{\text{OMR}})$ that maximizes the score $s(u, v, w)$.

4.2 Coupled CTR Algorithm

We next present our coupled CTR algorithm for the score-based CTR recommendation problem, with performance guarantees.

Challenges. Integrating the execution of OMRs and TIEs is non-trivial. Applying OMRs too early wastes work on anchor candidates that are later pruned by TIE refinements, while delaying OMR evaluation reduces efficiency. The challenge is further compounded by ranking costs: naively recomputing scores over $O(|V|)$ alternative candidates after each refinement step is costly.

To address these challenges, we tightly couple TIEs and OMRs

in a single fixpoint over a shared candidate space, yielding a joint-style evaluation rather than post-processing. CTR further updates alternative scores incrementally to avoid redundant recomputation.

Overview. As illustrated in Figure 4, the coupled CTR algorithm processes TIEs in rounds, each consisting of two steps:

- (1) *Rule-group construction.* For each TIE $\varphi_T = Q_T[x_0, y_0](X_T \rightarrow \text{Rec}(x_0, y_0)) \in \Sigma_{\text{TIE}}$, we construct a rule group $\mathcal{H}(\varphi_T)$ by collecting all OMRs in Σ_{OMR} whose constraints on (x_0, y_0) are compatible with those of φ_T . Each group maintains a shared candidate map to support joint TIE–OMR refinement.
- (2) *Coupled refinement.* For each rule group $\mathcal{H}(\varphi_T)$, we iteratively refine its candidate map using two operators: (a) a *local refinement* that enforces the constraints of an individual OMR, and (b) a *group refinement* that aggregates support across rules via existential semantics. The process terminates at a fixpoint.

Algorithm. We now present algorithm CTR in Figure 5. Given a graph $G(V, E)$, a user u , a set Σ_{OMR} of OMRs, a set Σ_{TIE} of TIEs, $\mathcal{M}_{\text{dis}}$, $\mathcal{M}_{\text{aggr}}$ and λ , CTR returns a set $I(u)$ of anchors and a set $I_d(u)$ of alternatives by computing the matches for TIEs and OMRs in a single yet shared candidate map via a coupled fixpoint procedure.

Initially, CTR initializes an empty score set A_v for each item vertex v (Line 1), used to incrementally maintain ranking scores of alternatives w.r.t. v . It then processes TIEs in Σ_{TIE} in rounds to get converged candidates for both anchors and alternatives (Lines 2–11).

For each $\varphi_T \in \Sigma_{\text{TIE}}$, it builds a rule group H of compatible OMRs via `constructGroup`, inducing a joint pattern Q^H and its rooted DAG Q_D^H for coupled evaluation (Line 3). It initializes a shared candidate map C^H and sets the group operator $F_H \leftarrow C^H$ (Line 4).

The algorithm then performs coupled refinement until convergence. For each OMR $\varphi \in H$, it applies procedure `localRefinement` to enforce φ ’s preconditions on F_H using Q_D^H (Line 7). For every candidate pair $(v, w) \in F_{\varphi}(y_0, w_0)$, it incrementally updates the ranking score $s(u, v, w)$ via `updateScore` and records w in A_v (Lines 9–10). After processing all rules in H , it applies `groupRefinement` to update F_H (Line 11). Once a fixpoint is reached, it extracts anchor items from $F_H(y_0)$ and, for each anchor v , selects the highest-scoring alternatives from $F_H(w_0)$ to form $I_d(u)$ (Lines 12–13).

Example 4: We illustrate CTR for the user u_1 on the graph in Figure 1(a). Suppose that Σ_{TIE} has one TIE φ_T , and Σ_{OMR} has two OMRs φ_4 and φ_5 , as shown in Figures 6(a), (b) and (c), respectively.

It first builds a rule group H containing φ_T and φ_5 , where φ_4 is excluded because its constraint on x_0 .age conflicts with that in φ_T . It also induces a joint pattern Q^H and its rooted DAG Q_D^H as shown in Figure 6(d). Then, it performs a local refinement by enforcing φ_T ’s constraints, making $C^H(x_0) = \{u_1\}$, $C^H(y_0) = \{v_1\}$, $C^H(x_1) = \{u_2\}$, $C^H(x_2) = \{v_2\}$, $C^H(x_3) = \{o_1\}$, $C^H(x_4) = \{t_1\}$, $C^H(w_0) = \{v_2\}$ and $C^H(x_5) = \{o_2\}$. For (v_1, v_2) , it updates the score accordingly. Next, it performs a local refinement by φ_5 , making no changes and

Input: $G(V, E)$, u , a set Σ_{OMR} of OMRs, a set Σ_{TIE} of TIEs, $\mathcal{M}_{\text{dis}}$, $\mathcal{M}_{\text{aggr}}$ and λ .
Output: a set $I(u)$ of anchor items and a set $I_d(u)$ of alternative items.

1. $A_v := \emptyset$ for each item vertex $v \in V$; $I(u) := \emptyset$; $I_d(u) := \emptyset$;
2. **for each** φ_T in Σ_{TIE} **do**
3. $H, Q^H, Q_D^H := \text{constructGroup}(u, \varphi_T, \Sigma_{\text{OMR}})$;
4. $C^H := \text{InitCand}(Q^H, u)$; $F_H := C^H$;
5. **while** F_H does not converge **do**
6. **for each** $\varphi \in H$ **do**
7. $F_\varphi := \text{localRefinement}(F_H, Q_D^H, \varphi)$;
8. **for each pair** $(v, w) \in F_\varphi(y_0, w_0)$ **do**
9. $s(u, v, w) := \text{updateScore}(u, v, w, \mathcal{M}_{\text{dis}}, \mathcal{M}_{\text{aggr}}, \lambda)$;
10. $A_v := A_v \cup \{s(u, v, w)\}$;
11. $F_H := \text{groupRefinement}(\{F_\varphi | \varphi \in H\})$;
12. $I(u) := I(u) \cup F_H(y_0)$;
13. $I_d(u) := I_d(u) \cup F_H(w_0)$ with the highest ranking score;
14. **Return** $I(u)$ and $I_d(u)$;

Figure 5: CTR algorithm.

so the group refinement reaches a fixpoint. It then terminates. $\square$

The procedures used in the CTR algorithm are as follows.

Procedure constructGroup. Given a user u , a set Σ_{OMR} of OMRs and a TIE $\varphi_T = Q_T[x_0, y_0](X_T \rightarrow \text{Rec}(x_0, y_0)) \in \Sigma_{\text{TIE}}$, this procedure constructs (a) a *rule group* $H = \mathcal{H}(\varphi_T)$ tailored for φ_T and induces (b) a *joint pattern* Q^H for this group H , followed by building (c) a *rooted DAG* Q_D^H of this joint pattern Q^H , as follows.

(a) *Rule group.* For each TIE φ_T , we construct a rule group

$$\mathcal{H}(\varphi_T) = \{\varphi_T\} \cup \Sigma_{\text{OMR}}^{\varphi_T},$$

where $\Sigma_{\text{OMR}}^{\varphi_T} \subseteq \Sigma_{\text{OMR}}$ consists of OMRs whose constraints on the pivot variables (x_0, y_0) are compatible with φ_T . An OMR $\varphi = Q_\varphi[x_0, y_0, w_0](X_\varphi \rightarrow \text{Rec}_d(x_0, w_0))$ is included in $\Sigma_{\text{OMR}}^{\varphi_T}$ if its constraints on x_0 are satisfied by user u , i.e., $L_G(u) = L_{Q_\varphi}(x_0)$, $\deg_G(u) \geq \deg_{Q_\varphi}(x_0)$, and u satisfies all constant predicates on x_0 in X_φ , where $\deg$ denotes the degree in G and Q_φ , respectively.

(b) *Joint pattern.* Given a rule group H , we construct a *joint pattern*

$$Q^H = \bigcup_{\varphi \in H} Q_\varphi,$$

where Q^H consists of variable set $V(Q^H)$ and edge set $E(Q^H)$. It encodes both the TIE topology for anchor selection and the OMR topology for generating alternatives for the same pivot pair (x_0, y_0) .

(c) *Rooted DAG.* We construct a rooted DAG Q_D^H from Q^H by treating the user variable x_0 as the root. We perform a BFS traversal from x_0 and orient edges from higher to lower levels, ensuring that Q_D^H is acyclic and that only x_0 has no incoming edges (see more in [4]).

Procedure InitCand. It initializes a shared candidate map

$$C^H = \{C^H(z) \subseteq V \mid z \in \text{Var}(Q^H)\}$$

for user u and group H . It sets $C^H(x_0) = \{u\}$, and for each $z \neq x_0$,

$$C^H(z) = \{v \in V \mid v \text{ satisfies the unary constraints of } z\}.$$

A vertex v is included in $C^H(z)$ if v satisfies all unary predicates on z in X^H , e.g., same label, $z.A \oplus c$ and ML predicates (see more in [4]).

Procedure localRefinement. Given the current candidate map C^H and a rule $\varphi(Q, X) \in H$, this procedure refines C^H to F_φ by enforcing (a) topological consistency with Q and (b) the constraints in X . It performs a forward pass and a backward pass over the DAG Q_D^H .

- *Forward pass.* For each variable $z \in V_Q$, a vertex v remains in $F_\varphi(z)$ iff (a) $v \in C^H(z)$; (b) for each child z_c of z in Q_D^H , there is an adjacent $v_c \in F_\varphi(z_c)$; and (c) for each variable predicate $z.A \oplus z_p.B$ in X , there is $v_p \in F_\varphi(z_p)$ such that $v.A \oplus v_p.B$ holds.

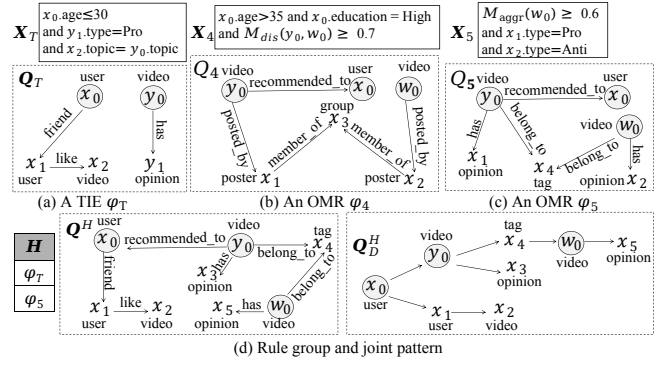


Figure 6: A running example of CTR.

- *Backward pass.* Traversing the reverse Q_D^{-1} of graph Q_D^H , a vertex v remains in $F_\varphi(z)$ iff $v \in C^H(z)$ and, for every parent z_p of z , there exists an adjacent vertex $v_p \in F_\varphi(z_p)$.

Unlike prior dynamic-programming approaches [36], we do not iterate these passes for a single rule. Instead, refinement is coordinated at the group level, allowing convergence to be determined over H as a whole and avoiding redundant per-rule iteration.

Procedure updateScore. When multiple OMRs (or matches of the same OMR) produce the same alternative w for an anchor v , updateScore aggregates their contributions to the ranking score $s(u, v, w)$ incrementally, avoiding recomputation from scratch.

Both the topic proximity $\text{sim}_T(v, w)$ and the acceptability term $s_{\text{acc}}(u, w)$, which depend only on $\mathcal{M}(u, w)$ and $\mathcal{M}_{\text{aggr}}(w)$, are fixed once computed. Only the content dissimilarity component $s_{\text{dis}}(u, v, w, \varphi)$ varies across OMRs. Thus, $s(u, v, w)$ is updated by adding the incremental contribution of the current φ .

Specifically, for each $(v, w) \in F_\varphi(y_0, w_0)$, if w already appears in A_v , we increase $s(u, v, w)$ by $\lambda \cdot \gamma \cdot \Delta_\varphi(u, v, w)$; otherwise, we compute $s(u, v, w) = (1 - \lambda) s_{\text{acc}}(u, w) + \lambda \cdot \text{sim}_T(v, w) \cdot \frac{1}{|\Sigma|} \cdot s_{\text{dis}}(u, v, w, \varphi)$. The correctness of the process is established by Lemma 1 (see [4]).

Lemma 1: Given user u and an anchor v , for any alternative item w at which at least one OMR is applicable, UpdateScore is correct to compute the ranking score $s(u, v, w)$ of w w.r.t. v . $\square$

Procedure groupRefinement. Rather than requiring all rules to hold simultaneously, groupRefinement adopts *existential rule support*: a candidate vertex for variable $z \in Q^H$ is retained if it is supported by at least one rule in H via a valid local refinement. This matches the witness-based semantics of TIEs and OMRs. Formally, given local refinements F_φ for each $\varphi \in H$, the updated candidate map is

$$F_H = \bigcup_{\varphi \in H} F_\varphi(C^H).$$

Starting from C^H , the algorithm iteratively applies

$$F_H^{i+1} := \text{groupRefinement}(F_H^i)$$

until convergence. Rules in H may be applied in any order. Once F_H reaches a fixpoint, we extract anchors from $F_H^\infty(y_0)$ that satisfy the TIE precondition X_T . For each anchor v , we choose the highest-scoring alternatives from $F_H^\infty(w_0)$ supported by some rule $\varphi \in H$.

Termination and confluence. By Lemma 2 below, each local refinement operator is monotone, and thus the group operator F_H is monotone over the finite lattice of candidate maps, ensuring convergence to a fixpoint. By Theorem 3 and Lemma 2, the coupled evaluation is sound and complete w.r.t. the sequential CTR baseline, yielding identical anchor-alternative pairs; the fixpoint is thus

unique and independent of rule application order. Moreover, shared monotone refinement avoids redundant candidate exploration (Theorem 4), reducing cost while retaining the same outcomes.

Lemma 2: For any group H and candidate maps C^H, C'^H such that $C'^H(z) \subseteq C^H(z)$ for all z , we have $F_H(C'^H) \subseteq F_H(C^H)$. Hence the coupled iteration converges to a unique fixpoint F_H^∞ . $\square$

Proof sketch: We first prove that both local refinement F_ϕ and group refinement F_H are monotone. Since candidate sets are finite and monotonically shrink until stabilization, the iteration converges in finitely many steps to a unique fixpoint. $\square$

Theorem 3: For any user u , TIE ϕ_T and rule group H , let $\text{Seq}(u, H)$ be the set of anchor-alternative pairs (v, w) produced by the sequential algorithm, and $\text{Cpl}(u, H)$ be the set of pairs extracted from the fixpoint F_H^∞ of our coupled algorithm. Then $\text{Seq}(u, H) = \text{Cpl}(u, H)$. $\square$

Proof sketch: For the soundness, we show that any $(v, w) \in \text{Cpl}(u, H)$ satisfies the same TIE and OMR conditions as in the sequential algorithm. We prove the completeness by showing that any $(v, w) \in \text{Seq}(u, H)$ admits compatible TIE and OMR witnesses whose union is preserved by the coupled fixpoint. $\square$

Theorem 4: For any user u and rule group H , let $\mathcal{N}_{\text{Seq}}$ be the set of candidate pairs explored by the sequential CTR algorithm, and $\mathcal{N}_{\text{Cpl}}$ be those by the coupled algorithm. Then $|\mathcal{N}_{\text{Cpl}}| \leq |\mathcal{N}_{\text{Seq}}|$. $\square$

Proof sketch: We show the stronger inclusion $\mathcal{N}_{\text{Cpl}} \subseteq \mathcal{N}_{\text{Seq}}$, indicating that the coupled algorithm only prunes candidates earlier than the sequential pipeline and does not introduce new ones. $\square$

Complexity analysis. The time complexity of CTR is $O(|\Sigma_{\text{TIE}}| \cdot (|\Sigma_{\text{OMR}}| + |Q^H| + t|H||Q|_{\max}|G|))$, where t is the number of group refinements to reach a fixpoint, and $|Q|_{\max}$ is the maximum pattern size among rules in H . For each TIE $\phi_T \in \Sigma_{\text{TIE}}$: (a) constructing the rule group costs $O(|\Sigma_{\text{OMR}}|)$ by checking pivot compatibility; (b) initializing the candidate map requires $O(|Q^H|)$; and (c) iterative group refinement costs $O(t|H||Q|_{\max}|G|)$, since each local refinement takes $O(|Q||G|)$ time following [27].

5 TOP-K RECOMMENDATION

This section develops TopK, the algorithm of OpenMind to make top- k recommendations along with suggested diverse viewpoints. The top- k recommendation problem is formulated as follows.

- **Input:** $G, \Sigma_{\text{TIE}}, \Sigma$ and u as in CTR (Section 4), and an integer k .
- **Output:** a set $K(u)$ of top- k ranked anchors with the highest recommendation strength, and a set $K_d(u)$ of top- k alternative items with the highest aggregated ranking scores w.r.t. $K(u)$.

Here for an alternative item w , its aggregated ranking score w.r.t. the selected top- k anchors in $K(u)$ is defined as

$$s(u, w|K(u)) = \max_{v \in K(u)} s(u, v, w).$$

The use of aggregated ranking score reflects a key difference from CTR. While CTR measures the relevance of an alternative to a given anchor, TopK assesses alternatives w.r.t. the entire top- k anchor set.

Challenge. A brute-force method first generates all anchors and alternative items using our CTR algorithm, and then selects the top- k results via sorting. This approach is inefficient, as it performs many evaluations for candidates that will never appear in the top- k sets. Early pruning via upper bounds is a natural idea, but is

difficult in our setting. The problem involves two coupled objectives: anchor and alternative recommendation, with asymmetric scoring functions, making it hard to derive tight bounds. Worse yet, the two tasks are interdependent: the anchors in $K(u)$ determine the feasible and high-scoring alternatives in $K_d(u)$. This dependency makes it challenging to optimize both objectives simultaneously without falling back to an inefficient two-stage pipeline.

To address these challenges, we propose an efficient algorithm TopK. It conceptually separates anchor recommendation (driven by TIEs) and alternative recommendation (by OMRs) into two *weakly coupled* stages. Specifically, we first identify the top- k anchors $K(u)$, and then compute the top- k alternatives $K_d(u)$ w.r.t. $K(u)$ by restricting the candidate space, aggregating scores over the anchors and using *rule-level upper bounds*, avoiding unnecessary computations for candidates that cannot appear in either top- k set.

5.1 Top- k Anchor Selection

We begin with the ranking strategy for selecting the top- k anchors.

Anchor ranking strategy. We rank anchors using the *updated* recommendation strength that combines the base prediction $\mathcal{M}_{\text{rec}}$ with TIE-based refinement. The score $\mathcal{M}_{\text{rec}}(u, v)$ estimates how likely user u prefers item v (Section 2). We handle items in two cases w.r.t. a fuzzy area $[\delta_L, \delta_H]$: (a) If $\mathcal{M}_{\text{rec}}(u, v) \geq \delta_H$, the item is directly retained as an anchor candidate. (b) If $\mathcal{M}_{\text{rec}}(u, v) \in [\delta_L, \delta_H]$, we apply TIEs at (u, v) . If some $\phi \in \Sigma_{\text{TIE}}$ derives $\text{Rec}(u, v)$, the score of v is raised to δ_H ; otherwise it is reduced to δ_L .

Anchors are then selected as the top- k items under the updated strengths. When multiple items receive the same adjusted value, their relative order follows the original ranking of $\mathcal{M}_{\text{rec}}$.

Validation. Applying all TIEs to each fuzzy-range item is costly: (1) Many such items will never enter the top- k anchors, yet validating them wastes computation. (2) A single TIE may validate multiple anchors, so evaluating rules per item repeats the same refinements.

To address this, we adopt a *batch validation* strategy. (1) Items are scanned in descending $\mathcal{M}_{\text{rec}}$ by the order-preservation property in Lemma 5 (see [4] for a proof). (2) TIEs are evaluated on candidate batches rather than individually, avoiding redundant refinements.

Lemma 5: Given a user u , for any two items v_1 and v_2 with $\mathcal{M}_{\text{rec}}(u, v_1) > \mathcal{M}_{\text{rec}}(u, v_2)$, if both are selected into top- k set, their relative ordering remains the same, i.e., v_1 is ranked before v_2 . $\square$

Batch validation via TIEs aggregation. To validate items in batches, we maintain a sliding window over the $\mathcal{M}_{\text{rec}}$ -sorted list H_{rec} . In each iteration, we select a contiguous slice V_{sel} of promising anchors, initially the top k positions. For this batch, we aggregate only the TIEs in Σ_{TIE} whose constraints on y_0 are compatible with at least one $v \in V_{\text{sel}}$. Formally, the unified set $\Sigma_{\text{TIE}}^s(V_{\text{sel}})$ is defined:

$$\Sigma_{\text{TIE}}^s(V_{\text{sel}}) = \{\phi_T \in \Sigma_{\text{TIE}} \mid C^{\phi_T}(y_0) \cap V_{\text{sel}} \neq \emptyset\},$$

where $C^{\phi_T}(y_0)$ is the initial candidate map at y_0 under ϕ_T . Intuitively, a TIE ϕ_T is selected iff there exists an item $v \in V_{\text{sel}}$ such that v satisfies the unary constraints of y_0 in ϕ_T .

Then, we run coupled refinement over this unified set $\Sigma_{\text{TIE}}^s(V_{\text{sel}})$ like in CTR, which simultaneously validates all anchors in V_{sel} . After processing $\Sigma_{\text{TIE}}^s(V_{\text{sel}})$, we discard those in V_{sel} that remain unsupported, and slide the window forward to include the next high- $\mathcal{M}_{\text{rec}}$ items. This process repeats until k anchors are determined.

Input: $G, u, \mathcal{M}_{\text{rec}}, \Sigma_{\text{TIE}}, \delta_H$ and k .

Output: a set $K(u)$ of top- k ranked anchor items

```

1.  $H_{\text{rec}} := \text{Sort}(V_{\text{item}}, \mathcal{M}_{\text{rec}})$ ;  $K(u) := \emptyset$ ;  $i_b := 0$ ;  $i_e := k$ ;
2. for each  $v \in H_{\text{rec}}$  do
3.   if  $|K(u)| < k$  and  $\mathcal{M}_{\text{rec}}(u, v) \geq \delta_H$  then
4.      $K(u) := K(u) \cup \{v\}$ ;  $i_b := i_b + 1$ ;
5.   else break;
6. while  $|K(u)| < k$  do
7.    $V_{\text{sel}} := \{H_{\text{rec}}(j) \mid i_b \leq j \leq i_e\}$ ;
8.    $\Sigma_{\text{TIE}}^s(V_{\text{sel}}) := \{\varphi_T \in \Sigma_{\text{TIE}} \mid C^{\varphi_T}(y_0) \cap V_{\text{sel}} \neq \emptyset\}$ ;
9.    $V_{\text{valid}} := \text{Refinement}(V_{\text{sel}}, \Sigma_{\text{TIE}}^s(V_{\text{sel}}, u)$ ;
10.   $K(u) := K(u) \cup V_{\text{valid}}$ ;
11.   $i_b := i_b + 1$ ;  $i_e := i_e + k - |V_{\text{valid}}|$ ;
12. Return  $K(u)$ ;
```

Figure 7: Top- k anchor selection algorithm TopKAnchor

Algorithm TopKAnchor. We now present TopKAnchor, the top- k anchor selection algorithm, in Figure 7. Given a graph G , a user u , a set Σ_{TIE} of TIEs, the base strength $\mathcal{M}_{\text{rec}}(u, v)$ for each item v and a constant k , TopKAnchor returns the top- k anchor set $K(u)$.

TopKAnchor first sorts items by descending $\mathcal{M}_{\text{rec}}$ to form H_{rec} , and initializes $K(u)$ and the sliding-window indices i_b and i_e (Line 1). It then scans items whose base score exceeds δ_H and inserts them into $K(u)$ while $|K(u)| < k$ (Lines 2–5). If fewer than k anchors are obtained, the algorithm performs iterative batch validation over the remaining items (Lines 6–11). In each iteration, it selects a candidate batch $V_{\text{sel}} = H_{\text{rec}}[i_b, i_e]$ (Line 7), constructs the unified rule set $\Sigma_{\text{TIE}}^s(V_{\text{sel}})$ (Line 8), and invokes Refinement to derive anchors V_{valid} supported by at least one TIE (Line 9). The validated anchors are added to $K(u)$, and the window is advanced by setting $i_b = i_b + 1$ and $i_e \leftarrow i_e + k - |V_{\text{valid}}|$ for the next iteration (Lines 10–11).

Procedure Refinement performs forward and backward passes for each rule in $\Sigma_{\text{TIE}}^s(V_{\text{sel}})$ like in CTR until convergence. By initializing the candidate maps $C(x_0) = \{u\}$ and $C(y_0) = V_{\text{sel}}$, it restricts the candidate space, reducing the cost of the refinement passes.

Example 5: Assume graph G has five item vertices and $k = 2$. Initially, TopKAnchor sorts items as $H_{\text{rec}} = \{v_1, \dots, v_5\}$, and none has base strength above δ_H , thus batch validation starts. In the first round, the window $[1, 2]$ gives $V_{\text{sel}} = \{v_1, v_2\}$ and $\Sigma_{\text{TIE}}^s(V_{\text{sel}}) = \{\varphi_1, \varphi_3\}$. Procedure Refinement evaluates these rules with $C(x_0) = \{u\}$ and $C(y_0) = \{v_1, v_2\}$. Only v_1 is validated, i.e., $V_{\text{valid}} = \{v_1\}$. The algorithm updates $K(u)$ and advances the window to $i_b = 3$ and $i_e = 3$. The process continues until k anchors are found. $\square$

Complexity. TopKAnchor takes $O(|V_{\text{item}}| \log |V_{\text{item}}|)$ time to sort the items. For each batch V_{sel} , the refinement over $\Sigma_{\text{TIE}}^s(V_{\text{sel}})$ takes $O(|\Sigma_{\text{TIE}}^s(V_{\text{sel}})| \cdot |Q| \cdot |C_{\varphi}|_{\text{max}})$ time, where $|C_{\varphi}|_{\text{max}}$ is the maximum size of the candidate map C_{φ} over all φ . This is because each refinement removes at least one vertex from a candidate map C_{φ} . Hence the overall complexity of TopKAnchor is $O(|V_{\text{item}}| \log |V_{\text{item}}| + |\Sigma_{\text{TIE}}^s(V_{\text{sel}})| \cdot |Q| \cdot |C_{\varphi}|_{\text{max}})$. When $C_{\varphi}(y_0)$ is restricted to items in V_{sel} , the candidate space gets much smaller, further improving efficiency.

5.2 Coupled Top- k Algorithm

We now present TopK to identify the top- k anchors and alternatives.

Overview. Given a user u , TopK first identifies the top- k anchors $K(u)$ using TopKAnchor. It then finds a global top- k alternative set $K_d(u)$ induced by the selected anchors in $K(u)$, by applying the same local and group refinements as CTR over the OMRs in Σ_{OMR} .

Input: $G, u, \Sigma_{\text{OMR}}, \Sigma_{\text{TIE}}, \mathcal{M}_{\text{rec}}, \mathcal{M}_{\text{dis}}, \mathcal{M}_{\text{aggr}}, \lambda, \delta_H$ and k .

Output: a set $K(u)$ of anchor items and a set $K_d(u)$ of alternative items.

```

1.  $\Psi := \emptyset$ ;  $K(u) := \emptyset$ ;  $K_d(u) := \emptyset$ ;
2.  $K(u) := \text{TopKAnchor}(u, \Sigma_{\text{TIE}}, \mathcal{M}_{\text{rec}}, \delta_H, k)$ ;
3.  $H, Q^H, Q_D^H := \text{constructGroup}(u, K(u), \Sigma_{\text{OMR}})$ ;
4.  $C^H := \text{InitCand}(Q^H, u)$ ;  $F_H := C^H$ ;
5. Initialize  $\text{best}_w$  and  $\hat{s}(u, w)$  for each  $w \in F_H(w_0)$ ;
6. for each  $(v, w) \in F_H(y_0, w_0)$  do
7.   Initialize  $\hat{s}(u, v, w)$  by the current rule-level upper bounds;
8.   if  $\hat{s}(u, v, w) \geq \hat{s}(u, w)$  then  $\hat{s}(u, w) := \hat{s}(u, v, w)$ ;
9.   while  $F_H$  does not converge do
10.    for each  $\varphi \in H$  do
11.       $F_{\varphi} := \text{localRefinement}(F_H, Q_D^H, \varphi)$ ;
12.      for each  $w \in F_{\varphi}(w_0)$  do
13.         $\tau :=$  the  $k$ -th highest aggregated score in  $\Psi$ ;
14.        if  $\hat{s}(u, w) < \tau$  then Remove  $(w)$  from  $F_{\varphi}(w_0)$ ; continue;
15.        for each  $v$  such that  $v \in F_{\varphi}(y_0)$  and  $(v, w) \in F_{\varphi}(y_0, w_0)$  do
16.           $s(u, v, w) := \text{updateScore}(u, v, w, \mathcal{M}_{\text{dis}}, \mathcal{M}_{\text{aggr}}, \lambda)$ ;
17.          if  $s(u, v, w) > \text{best}_w$  then
18.             $\text{best}_w := s(u, v, w)$ ;
19.            if  $\text{best}_w > \tau$  then Update  $\Psi$  and  $\tau$  by  $w$ ;
20.          Update (tighter)  $\text{UB}_{\varphi}(u, v)$  and affected  $\hat{s}(u, w')$  based on  $F_{\varphi}$ ;
21.       $F_H := \text{groupRefinement}(\{F_{\varphi} \mid \varphi \in H\})$ ;
22.  $K_d(u) := \Psi$ ;
23. Return  $K(u)$  and  $K_d(u)$ ;
```

Figure 8: TopK algorithm.

To avoid naively running local refinements for every OMR $\varphi \in \Sigma_{\text{OMR}}$ at each top- k anchor v , TopK adopts an early pruning strategy based on a *candidate-level upper bound*. TopK maintains (a) a global max-heap Ψ storing the current top- k alternatives and (b) a threshold τ , the k -th largest aggregated score in Ψ . Before performing local refinement over the rule group H , the algorithm computes a global upper bound $\hat{s}(u, w)$ for each candidate alternative w . If $\hat{s}(u, w) < \tau$, w cannot enter the top- k alternatives and is pruned; otherwise, TopK applies local refinements on rules in H and updates the scores of the remaining candidates. As in CTR, the algorithm iteratively performs group refinement over H to progressively shrink the candidate map and tighten the upper bounds, until a fixpoint is reached.

Below we define the upper bound, followed by details of TopK.

Candidate-level upper bound. Given a user u , the top- k anchor set $K(u)$, and a rule group H , for each candidate alternative w , we define its global upper bound by considering all anchors in $K(u)$:

$$\hat{s}(u, w) = \max_{v \in K(u)} \hat{s}(u, v, w),$$

where $\hat{s}(u, v, w)$ is the upper bound w.r.t. an anchor v , defined as:

$$\hat{s}(u, v, w) = \lambda \cdot \frac{1}{|H|} \sum_{\varphi \in H} \text{UB}_{\varphi}(u, v) + (1 - \lambda) \cdot s_{\text{acc}}(u, w),$$

where $\text{UB}_{\varphi}(u, v)$ is a rule-level upper bound, defined as the maximum (possible) ranking score of an *arbitrary* alternative w found by φ at (u, v) pair. Formally, we define $\text{UB}_{\varphi}(u, v)$ by considering the increment contributed by φ to an arbitrary w w.r.t. (u, v) :

$$\text{UB}_{\varphi}(u, v) = \hat{\gamma}(v) \cdot \Delta_{\varphi}^{\text{max}},$$

where $\Delta_{\varphi}^{\text{max}}$ denotes the maximum score increment contributed by an OMR φ , i.e., $\Delta_{\varphi}^{\text{max}} = \max_{w \in F_{\varphi}(w_0|v)} \Delta_{\varphi}(u, v, w)$, and $\hat{\gamma}(v) = \max_{w \in F_{\varphi}(w_0|v)} \gamma(v, w)$. Here $F_{\varphi}(w_0|v)$ is the set of candidate alternatives at variable w_0 after the local refinement F_{φ} , restricted to $F_{\varphi}(y_0) = v$. As refinement proceeds, some candidates w may be removed from $F_{\varphi}(w_0|v)$, allowing the rule-level upper bound to be progressively tightened (see [4]).

Intuitively, when $UB_\varphi(u, v)$ decreases by Δ_{UB_φ} , the candidate-level upper bound $\hat{s}(u, v, w)$ is reduced by $\lambda \cdot \frac{1}{|H|} \cdot \Delta_{UB_\varphi}$. The following result shows that our upper bound is sound for early pruning alternatives that cannot enter the top- k set of anchor v (see [4]).

Theorem 6: *Given a user u and top- k anchors in $K(u)$, the global upper bound $\hat{s}(u, w)$ is sound for pruning non-top- k alternatives.* $\square$

Proof sketch: We first prove that for any w at any iteration, its score $s(u, w|K(u)) \leq \hat{s}(u, w)$. Based on that, we show that pruning a candidate alternative w is safe due to the monotonicity of upper bounds and the top- k threshold τ during refinements. $\square$

Algorithm. We present algorithm TopK in Figure 8. It returns a top- k anchor set $K(u)$ and the top- k alternatives for each $v \in K(u)$.

Initially, TopK initializes an empty top- k heap Ψ , and empty anchor and alternative sets $K(u)$ and $K_d(u)$ (Line 1). It invokes TopKAnchor to obtain the top- k anchors $K(u)$ (Line 2), and constructs a compatible rule group H from Σ_{OMR} via constructGroup (Line 3), restricted to rules whose anchor constraints are satisfied by some $v \in K(u)$. Next, for each candidate alternative $w \in F_H(w_0)$, it initializes the aggregated score $best_w$ and global upper bound $\hat{s}(u, w)$ (Line 5). For each pair $(v, w) \in F_H(y_0, w_0)$, it initializes the candidate-level bound $\hat{s}(u, v, w)$ and updates $\hat{s}(u, w)$ (Lines 6–8).

The algorithm performs coupled refinement until convergence (Lines 9–21). For each OMR $\varphi \in H$, localRefinement enforces φ 's preconditions on F_H using Q_D^H (Line 11). For each $w \in F_\varphi(w_0)$, if $\hat{s}(u, w) < \tau$, w is pruned (Lines 13–14); otherwise, all $(v, w) \in F_\varphi(y_0, w_0)$ are enumerated to update $s(u, v, w)$ via updateScore, maintain $best_w$, and if $best_w > \tau$, update Ψ and τ (Lines 15–19). Afterward, rule-level bounds $UB_\varphi(u, v)$ and $\hat{s}(u, w)$ are updated based on F_φ (Line 20). After all $\varphi \in H$ are processed, groupRefinement updates F_H (Line 21). Upon convergence, the top- k alternatives in Ψ form $K_d(u)$ (Line 22; see [4]).

Example 6: Consider a user u with $k = 2$, $\lambda = 0.5$, and three candidate anchors v_1, v_2 and v_3 . TopKAnchor returns $K(u) = \{v_1, v_2\}$. Initially, $\Psi = \emptyset$ and $\tau = 0$. After local refinement with φ_1 , candidates w_1, w_2 and w_3 are generated. Processing w_1 , it is not pruned; (v_1, w_1) yields $s(u, v_1, w_1) = 0.6$, so $best_{w_1} = 0.6$ and $\Psi = \{(w_1, 0.6)\}$. Next, w_2 has $\hat{s}(u, w_2) = 0.533$ and is retained since $|\Psi| < k$; (v_1, w_2) gives $s(u, v_1, w_2) = 0.55$, updating $best_{w_2} = 0.55$ and $\Psi = \{(w_1, 0.6), (w_2, 0.55)\}$ with $\tau = 0.55$. In contrast, w_3 is pruned since $\hat{s}(u, w_3) = 0.5 < \tau$. As refinement proceeds, bounds tighten until group refinement reaches a fixpoint. $\square$

Complexity. TopK takes $O(T_{\text{Anchor}} + |\Sigma_{OMR}| + |Q^H| + t|H|c_{\text{match}})$ time in total, since (a) TopKAnchor takes T_{Anchor} time, (b) it takes $O(|\Sigma_{OMR}| + |Q^H|)$ time to construct the rule group and initialize the candidate map; and (c) iterative group refinement costs $O(t|H|c_{\text{match}})$, where c_{match} is the cost of a single local refinement in TopK. Note that $c_{\text{match}} \ll |Q|_{\max}|G|$, as many unnecessary candidates are pruned by the global upper bounds.

6 EXPERIMENTAL STUDY

Using real-life graphs, we experimentally evaluated OpenMind for the (a) quality of its recommendations and (b) the efficiency and scalability of its CTR and top- k algorithms. We also conducted a real-life case study to demonstrate the effectiveness of OpenMind.

Experimental setting. We start with the experimental setting.

Experimental data. We used three user-item interaction graphs in real-life. (1) Anime [1], a bipartite user-anime review network that contains 47K users, 16K anime and 130K reviews (edges). Anime was enriched with features from both user profiles and anime metadata, e.g., demographics and genres. (2) Steam [2], a large-scale bipartite user-game graph that contains 14.3M users, 2.6M games and 41M edges. We augmented the game vertices with extra metadata, e.g., pricing and tags. (3) Book [3], a large-scale bipartite user-book interaction network that contains 808K users, 1.56M books and 225M edges. Book has multiple user behaviors.

To evaluate the scalability, we implemented a graph generator to produce larger synthetic graphs of billion-scale. They were controlled by the number of vertices (up to 0.1B) and edges (up to 0.9B).

Following the prior work [27, 28, 84], we randomly picked 70%/10%/20% of interaction history of each user in each dataset as the training/validation/testing set. We treated each observed user-item interaction pair as a positive instance and did negative sampling to pair the user with an item that s/he did not interact before.

Baselines. We compared OpenMind with six state-of-the-art models for diversified recommendations: (1) DPP [15], using *Determinantal Point Process* for relevant and diverse recommendations; (2) AdpMMR [24], computing user diversity propensity via feature entropy; (3) PDGAN [88], a personalized DPP with learned kernel matrix; (4) RAPID [57], incorporating personalized diversity with topic representation during training; (5) RecLogic [27], generating only anchors by TIEs; and (6) Seq, which sequentially finds anchor-alternative pairs by first applying TIEs and then OMRs. For rule-based methods, we used DeepFM [35] as $\mathcal{M}_{\text{rec}}$.

We do not compare with influence maximization methods [11, 32, 41, 79] since they address network-wide polarization mitigation rather than personalized diverse recommendations. Other diversity-aware re-ranking models are excluded due to their poor performance compared with the selected models above [57].

Rules. We mined 436 (resp. 156), 567 (resp. 196), and 515 (resp. 206) OMRs (resp. TIEs) on Anime, Steam and Book, respectively.

Default parameters. By default, we set $\lambda = 0.6$ for score computations, and $k = 10$ for top- k recommendation. For the fuzzy area of $\mathcal{M}_{\text{rec}}$, δ_L and δ_H were automatically decided on validation datasets, yielding δ_H (resp. δ_L) as 0.61 (resp. 0.40), 0.68 (resp. 0.47) and 0.56 (resp. 0.31) on Anime, Steam and Book, respectively.

Evaluation metrics. We evaluated four commonly used metrics [25, 27, 29, 35, 82, 87, 94] for CTR recommendation. We evaluated each metric on n randomly selected testing users. We set $n = 1000$.

(1) *Area Under the ROC Curve (AUC)*. It measures recommendation accuracy, defined as the probability that a method returns a higher score for a random positive sample than for a random negative one.

(2) *Precision (Precis)*. It measures the ratio of true positives to all the decisions made by the recommendation method, defined as:

$$\text{Precis} = 1/n \sum_u \#TP_u / |\mathcal{R}_u|,$$

where $\mathcal{R}_u$ is the recommendation list for user u and $\#TP_u$ is the number of true positives at u . To be fair, we set $\mathcal{R}_u$ for OpenMind by merging two sets $I(u)$ and $I_d(u)$.

Methods	Anime				Steam				Book			
	AUC	Precis	ILD	FBP	AUC	Precis	ILD	FBP	AUC	Precis	ILD	FBP
AdpMMR	0.6876	0.3228	0.2441	0.1945	0.7589	0.3782	0.1274	0.1379	0.7829	0.3743	0.1387	0.2054
DPP	0.6621	0.321	0.2778	0.2634	0.7449	0.3712	0.1536	0.1591	0.7612	0.3688	0.1896	0.2369
PDGAN	0.6753	0.3202	0.2712	0.2516	0.7238	0.3468	0.1375	0.1408	0.7782	0.3569	0.1709	0.2237
RAPID	0.6882	0.3264	0.2818	0.2719	0.7519	0.3629	0.1539	0.1396	0.7894	0.3731	0.1678	0.2547
RecLogic	0.7292	0.3584	0.1918	0.1325	0.7802	0.4217	0.0764	0.0798	0.8163	0.4134	0.119	0.1681
Seq	0.6861	0.3282	0.3254	0.3869	0.7618	0.3727	0.1975	0.205	0.7882	0.3767	0.2412	0.3093
OpenMind	0.6861	0.3282	0.3254	0.3869	0.7618	0.3727	0.1975	0.205	0.7882	0.3767	0.2412	0.3093

Table 1: Effectiveness of CTR recommendations (the best results are in bold).

Methods	Anime				Steam				Book			
	Recall@10	NDCG@10	ILD@10	FBP@10	Recall@10	NDCG@10	ILD@10	FBP@10	Recall@10	NDCG@10	ILD@10	FBP@10
AdpMMR	0.5718	0.5101	0.2438	0.1973	0.3116	0.4948	0.1235	0.1348	0.4106	0.5301	0.1328	0.2075
DPP	0.55	0.5034	0.2741	0.2515	0.3079	0.4869	0.1493	0.1548	0.398	0.5258	0.1833	0.2296
PDGAN	0.5712	0.5078	0.2799	0.2508	0.2984	0.4862	0.1394	0.1442	0.4118	0.5432	0.1751	0.2262
RAPID	0.5897	0.5222	0.2836	0.2451	0.3048	0.4874	0.1562	0.1436	0.4185	0.5372	0.1826	0.2471
Seq	0.5834	0.5136	0.3233	0.3754	0.3193	0.4996	0.2139	0.2174	0.4199	0.5399	0.2562	0.2865
OpenMind	0.5834	0.5136	0.3233	0.3754	0.3193	0.4996	0.2139	0.2174	0.4199	0.5399	0.2562	0.2865

Table 2: Effectiveness of top- k recommendations where $k = 10$ (the best results are in bold).

(3) *Intra-List Diversity* (ILD). It evaluates the pairwise item diversity within one recommendation list, defined as follows:

$$ILD = \frac{1}{n} \sum_u \frac{2}{(|\mathcal{R}_u|)^2 - 1} \sum_{i,j \in \mathcal{R}_u} d_{ij},$$

where d_{ij} is the *Hamming distance* between two items (i, j) [47].

(4) *Filter Bubble Penetration* (FBP). It evaluates the proportion of recommended items outside users' echo chambers, defined as:

$$FBP = \frac{1}{n} \sum_u \frac{|\text{Out}(\mathcal{R}_u)|}{|\mathcal{R}_u|},$$

where $\text{Out}(\mathcal{R}_u)$ is the set of items outside u 's echo chamber, *i.e.*, items whose average similarity score to the items u historically interacted with is below threshold β (see [4] for how to determine β).

For top- k recommendation, we adopt two widely used metrics to evaluate accuracy [27, 57]. (1) *Recall@ k* [27], defined as the fraction of ground-truth positive items that appear in the top- k recommendation list. (2) *NDCG@ k* [44], which evaluates ranking quality by assigning higher weights to relevant items appearing at higher positions in the top- k list. For diversity, we measure *ILD@ k* and *FBP@ k* by instantiating $\mathcal{R}_u$ as the top- k recommendation list in their respective definitions. We vary $k \in \{10, 20, 30, 40, 50\}$.

Environment. The experiments were conducted on a single machine powered by 256GB RAM, 32 processors with Intel(R) Xeon(R) Gold 5320 CPU @2.20GHz and one NVIDIA TESLA V100 GPU with 32 GB memory. OpenMind is implemented mainly in C++. For space limit, we report results on some datasets; the results on the others are consistent. We run each experiment three times and report the average.

Experimental results. We next report our findings.

Exp-1: Effectiveness of CTR recommendation. As shown in Table 1, the accuracy of OpenMind (AUC and Precision) is comparable to SOTA baselines. This is expected, as it optimizes a multi-objective tradeoff between relevance and viewpoint diversity, but AUC/Precision capture only relevance. By explicitly incorporating dissimilar yet acceptable alternatives, OpenMind operates on a different point of the Pareto frontier, to improve diversity with minimal loss in accuracy. Hence, comparable accuracy indicates that OpenMind preserves relevance while effectively expanding the solution space beyond single-objective recommenders.

For diversity, OpenMind consistently outperforms all baselines. Its ILD is, on average, 54.08%, 24.31%, 34.92%, 29.18% and 110.28% higher than AdpMMR, DPP, PDGAN, RAPID and RecLogic over three datasets, respectively, up to 73.90%, 28.58%, 43.64%, 43.74% and 158.51%. Its FBP improves by 66.05%, 35.43%, 45.88%, 36.86% and

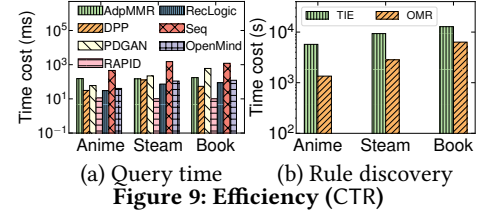
144.30% over these baselines. These come from differences in how diversity is modeled. Re-ranking methods AdpMMR, DPP, PDGAN and RAPID operate on pre-selected candidates and mostly diversify topics, and are thus limited in producing contrasting views. Moreover, AdpMMR, DPP and PDGAN focus on feature-level diversity of items, often ignoring user-item interaction structure, while RAPID models personalized topic diversity but still inherits a biased candidate pool from the base recommender. In contrast, OpenMind generates alternatives explicitly and models diversity at the view level by combining graph-structured interactions and feature signals via OMRs, enabling both broader coverage and stronger contrast. RecLogic is not designed for alternative recommendation and thus does not alter filter bubbles. While Seq achieves reasonable accuracy and diversity, it is much less efficient (see Exp-2).

Varying λ . We varied the trade-off parameter $\lambda \in [0, 1]$ from acceptability- to diversity-focused. As λ increases, the accuracy (AUC, Precis) decrease while the diversity (ILD, FBP) increase (Figure 10(a)), reflecting the relevance-diversity tradeoff. Thus, λ selects a point that balances preference alignment and dissimilarity. On Steam, $\lambda = 0.6$ strikes the best balance, with strong diversity and minimal accuracy loss. Similar results are observed for Anime (not shown) and Book (Figure 10(b)). This justifies the need for joint optimization rather than treating each objective in isolation.

Exp-2: Efficiency of CTR recommendation. We evaluated the average runtime per user of CTR in OpenMind. As shown in Figure 9(a), OpenMind is up to 13.7 $\times$ faster than Seq across all datasets, as it performs coupled refinement over a shared candidate map, avoiding redundant enumeration and refinement of candidates in Seq. On Steam, OpenMind takes only 0.11 seconds to find all anchor-alternative pairs, whereas Seq takes 1.5 seconds.

OpenMind is faster than AdpMMR and PDGAN by up to 3.76 $\times$ and 4.92 $\times$ over three datasets, respectively, since the baselines use sequential greedy selection to optimize topic diversity, while OpenMind incrementally updates scores in the coupled refinement process. It is comparable to DPP as DPP computes diversity for only a few selected items, instead of all items. It adds negligible overhead to RecLogic despite computing alternatives. It is slower than RAPID, which benefits from efficient end-to-end neural inference.

Time cost of rule discovery. We tested the efficiency of TIE and OMR discovery. As shown in Figure 9(b), OpenMind consistently mines OMRs faster than TIEs. On Anime, OpenMind takes only 0.37 hours to mine OMRs while it takes 1.59 hours for TIEs. The speedup comes from mining OMRs by extending TIEs, thus avoiding redundant



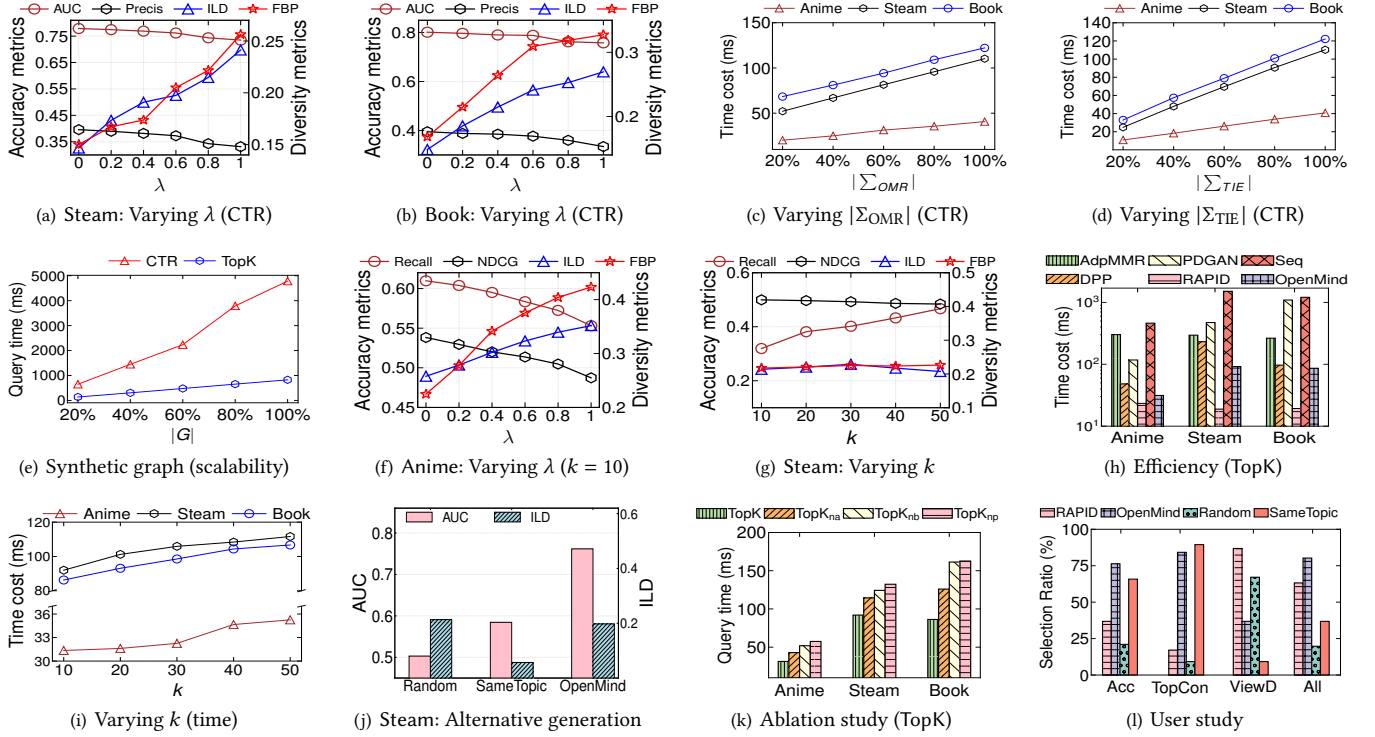


Figure 10: Performance evaluation

computations. Note that rule discovery is a one-time offline cost.

Varying $|\Sigma_{OMR}|$ and $|\Sigma_{TIE}|$. We varied the size of Σ_{OMR} (resp. $|\Sigma_{TIE}|$) in Figure 10(c) (resp. Figure 10(d)). As expected, OpenMind takes longer with more OMRs/TIEs, yet it remains efficient, handling hundreds of OMRs/TIEs within about 0.12 seconds on Book.

Varying $|G|$. We varied the scaling factor of G from 20% to 100% on the synthetic graphs. All methods take longer for a larger graph. In Figure 10(e), OpenMind is 7.37 $\times$ slower when $|G|$ is from 20% to 100%, indicating that its runtime grows steadily with graph size.

Exp-3: Effectiveness of TopK recommendation. Table 2 reports results for $k = 10$. On average over three datasets, OpenMind beats the baselines by 3.10%, 1.29%, 42.19% and 45.98% in Recall@10, NDCG@10, ILD@10 and FBP@10, respectively. On Book, e.g., its FBP is better than the best baseline RAPID by up to 15.94%, validating its effectiveness in breaking filter bubbles. Although RAPID has higher Recall@10 and NDCG@10 on Anime, its diversity is much worse: lower ILD@10 and FBP@10 by 14% and 53.16%, respectively.

Varying λ . We varied λ from 0 to 1 on Anime (Figure 10(f)). As λ increases, the diversity metrics ILD and FBP improve while Recall and NDCG slightly decrease, reflecting the diversity-acceptability tradeoff. Thus, λ controls the balance between diversity and accuracy; similar for Steam and Book (not shown).

Varying k . We varied k from 10 to 50. As shown in Figure 10(g), ILD and FBP on Steam remain stable, indicating that the ranking score consistently prioritizes more promising alternatives. Meanwhile, Recall@ k increases with k as more relevant items are included, while NDCG@ k remains stable, showing that top-ranked quality is preserved. Similar trends hold on Anime and Book (not shown).

Exp-4: Efficiency of top- k recommendation. We report the

average per-user runtime for top- k recommendation in Figure 10(h). On Book, OpenMind takes 86 ms on average. (1) OpenMind is much faster than DPP, AdpMMR, PDGAN and Seq, by up to 2.5 $\times$, 9.6 $\times$, 12.6 $\times$ and 16.4 $\times$, respectively. This is because the baselines rely on sequential greedy selection, while TopK in OpenMind employs multiple pruning strategies to avoid unnecessary computations. (2) OpenMind is slower than RAPID, which uses an end-to-end neural model to score all items and directly return the top- k results. However, RAPID incurs higher offline training cost.

Varying k . As shown in Figure 10(i) by varying k from 10 to 50 on Book, TopK incurs longer time as k increases due to the need to evaluate more candidates. On the largest graph Book, the runtime increases by only 20.4 ms as k increases from 10 to 50.

Varying $|G|$. We varied the scaling factor of the synthetic graph G from 20% to 100%. All methods take longer on larger graphs. In Figure 10(e), TopK is 5.99 $\times$ slower when $|G|$ is from 20% to 100%.

Varying $|\Sigma_{OMR}|$ and $|\Sigma_{TIE}|$. We varied the sizes of Σ_{OMR} and $|\Sigma_{TIE}|$. TopK takes longer with more OMRs or TIEs (not shown).

Exp-5: Ablation study. We conducted ablation studies to evaluate the contribution of each component in OpenMind.

Effectiveness of alternative generation. We compared OpenMind with two variants on generating alternatives for CTR recommendation: Random that randomly samples alternatives from all items, and SameTopic that randomly selects alternatives from items of the same topic as the anchor. In Figure 10(j), OpenMind achieves 30.33% and 247.71% higher AUC and ILD than SameTopic on Steam, respectively, verifying the effectiveness of viewpoint-aware alternative generation. Compared with Random, OpenMind improves the AUC by 51.45%, as Random selects items that are misaligned with the user’s interests, confirming the need for user acceptability.

CTR. We compared CTR in OpenMind with its variant CTR_{nu} that naively computes the score for each pair without using `updateScore`. CTR runs faster than CTR_{nu} by up to 10 $\times$ on Book, verifying the effectiveness of incrementally updating scores (not shown).

TopK. We compared TopK with three variants: (1) $TopK_{na}$, which enumerates all anchor-alternative pairs via CTR, ranks them by score, and returns the top- k results; (2) $TopK_{nb}$, which follows the same pipeline as TopK but replaces batch validation with per-item TIE evaluation; and (3) $TopK_{np}$, which adopts the same anchor selection as TopK but disables upper-bound pruning for alternatives.

In Figure 10(k), OpenMind is 1.56 $\times$ faster than all baselines on average when $k = 10$. It beats $TopK_{na}$, $TopK_{nb}$ and $TopK_{np}$ by up to 1.46 $\times$, 1.87 $\times$ and 1.89 $\times$, respectively, validating the effectiveness of TopK’s design choices, including weakly coupled stages, batch validation for anchor selection, and early pruning for alternatives.

Exp-6: User study. We conducted a user study to assess which recommendation is better. We compared OpenMind with RAPID, Random and SameTopic on Anime. We evaluated four metrics, *i.e.*, (1) *Acceptability* (Acc): Would the user accept the recommendations? (2) *Topic Consistency* (TopCon): Do the recommended items remain in the same topic? (3) *View Divergence* (ViewD): Do the recommended items present different viewpoints/perspectives? (4) *Overall* (All) that considers all above metrics simultaneously (more in [4]).

In total, we collected 76 valid responses. In Figure 10(l), 61 of the responses indicate that OpenMind achieves the best overall performance. Although RAPID and Random have higher ViewD than OpenMind as both of them provide across-topic items, they sacrifice the overall performance. Compared with RAPID (resp. Random), OpenMind achieves 39.48% (resp. 55.27%), 67.1% (resp. 75%) and 17.1% (resp. 60.52%) higher Acc, TopCon and All, respectively, verifying the effectiveness of balancing both acceptability and diversity in alternative generation. Compared with SameTopic, OpenMind improves ViewD by 27.63%, showing that the topic preservation alone is insufficient for exposing users to opposing views.

Summary. We find the following. (1) OpenMind provides high-quality CTR recommendation with diverse viewpoints; its ILD and FBP outperform prior methods by 35.62% and 46.06% on average, up to 110.28% and 144.30%, respectively, while retaining comparable accuracy. (2) OpenMind is efficient, taking 0.12s per user on average for CTR on a graph with 225M edges. (3) It is 13.7 $\times$ faster than Seq on average for CTR recommendation, justifying coupled refinement. (4) For top- k recommendation, on average, Recall@ k , NDCG@ k , ILD@ k and FBP@ k improve by 3.10%, 1.29%, 42.19% and 45.98%, respectively, up to 7.0%, 2.76%, 92.92% and 90.27%. (5) TopK outperforms all baselines except RAPID in efficiency, with up to 16.4 $\times$ speedup. (6) TopK is 1.72 $\times$ faster than $TopK_{np}$ on average, up to 1.89 $\times$, validating the effectiveness of pruning.

7 RELATED WORK

We categorize the related work as follows.

Filter bubbles. Prior work on the topic falls into two categories.

Re-ranking. Given a user u and an initial list $\mathcal{R}_u$ from $\mathcal{M}_{rec}$, these methods reorder $\mathcal{R}_u$ to improve diversity, typically at the topic level. Representative approaches include MMR [13], DPP [15], and neural

variants such as DESA [67] and SSD [43], all of which apply a uniform diversity strategy to all users; while personalized extensions (*e.g.*, AdpMMR [24], PDGAN [88], RAPID [57]) incorporate diversification with user preferences. However, these methods operate on pre-selected candidates and diversify topics, limiting their ability to introduce contrasting viewpoints within the same context. Other methods (*e.g.*, DLCM [6], SetRank [64], SRGA [66]) ignore diversity.

Balanced information exposure. Another line of work addresses polarization via influence propagation and network interventions [11, 32, 41, 61, 79], including cross-partisan edge construction [31] and user interface interventions [52, 53].

This work differs from prior approaches in two key aspects. (1) Diverse recommendation methods [15, 22, 23, 57] primarily focus on *topic-level* (or category-level) diversity, aiming to broaden user exposure across topics. In contrast, OpenMind targets *viewpoint diversity*, *i.e.*, exposing users to alternative or opposing perspectives on the same topic. (2) Prior work on information chambers [11, 31, 32, 52, 53, 57, 61, 79] either studies user reactions to opposing views [52, 53] or optimizes placement and targeting to maximize influence in information propagation [11, 31, 32, 61, 79]. However, these approaches operate at the population level and do not provide mechanisms for generating alternative viewpoints during recommendation for individual users.

Recommendation systems. A host of recommendation models have been developed, including: (1) *CF-based models* that leverage user-item interactions [12, 39, 48, 49, 56, 59, 75, 85], often enhanced by deep learning architectures such as MLP [18, 26], autoencoders [69, 73, 74], CNN [38, 78] and GNN [37, 85]; (2) *content-based models* that exploit auxiliary features (*e.g.*, images and text) to profile users [58, 81, 86]; (3) *hybrid models* that combine multiple strategies [5, 14, 17, 19, 76, 80, 95, 97]; and (4) *rule-based frameworks* that augment recommendation with logical reasoning, allowing arbitrary models to be plugged in, *e.g.*, RecLogic [27]. *However, these models primarily optimize accuracy and do not explicitly account for diversity.*

OpenMind differs from prior systems in three aspects. (1) To the best of our knowledge, it is the first effort to equip existing recommenders with the ability to identify alternative or opposing viewpoints. (2) It is a lightweight, plug-and-play framework that augments a traditional recommender with complementary viewpoints, helping mitigate bias without modifying the underlying model. (3) In particular, it extends RecLogic by supporting top- k recommendation for both anchor and alternative viewpoints.

8 CONCLUSION

We have studied how to augment recommender systems with alternative viewpoints to mitigate filter bubbles while preserving relevance. We have proposed OpenMind, a lightweight rule-based framework that integrates logical reasoning with ML signals to generate complementary recommendations. OpenMind supports both CTR and top- k settings, coupling anchor selection and alternative generation in a single inference pass. Our analytical and empirical results have shown that viewpoint-aware recommendation can be achieved efficiently without modifying underlying models, offering a practical path to mitigating filter bubbles in real-world systems.

Future work includes cost-based scheduling for OMR discovery and rule weighting to balance diversity and efficiency at scale.

REFERENCES

- [1] 2024. Anime Dataset with Reviews - MyAnimeList. <https://www.kaggle.com/datasets/marlesson/myanimelist-dataset-anime-profiles-reviews>.
- [2] 2024. Game Recommendations on Steam. <https://www.kaggle.com/datasets/antonkozyriev/game-recommendations-on-steam/>.
- [3] 2024. Goodreads Book Reviews. <https://www.kaggle.com/datasets/pyipahmad/goodreads-book-reviews1/>.
- [4] 2026. OpenMind: Query-Time Diversification via Logical Rules (full). https://github.com/hdqlkncso24jvf/OpenMind/blob/main/OpenMind_full_version.pdf.
- [5] Sajad Ahmadian, Nima Joorabloo, Mahdi Jalili, Majid Meghdadi, Mohsen Afsharchi, and Yongli Ren. 2018. A temporal clustering approach for social recommender systems. In *ASONAM*. IEEE, 1139–1144.
- [6] Qingyao Ai, Keping Bi, Jiafeng Guo, and W Bruce Croft. 2018. Learning a deep listwise context model for ranking refinement. In *SIGIR*. 135–144.
- [7] Heba Al-Mamouri and Wadhah R Baice. 2023. Predicting Content Popularity on Social Media: An Analytical Approach Using Regression Modeling. In *International Conference on Computing and Communication Networks*. Springer, 649–659.
- [8] Mingxiao An, Fangzhao Wu, Chuhan Wu, Kun Zhang, Zheng Liu, and Xing Xie. 2019. Neural news recommendation with long-and short-term user representations. In *ACL*. 336–345.
- [9] Vito Walter Anelli, Tommaso Di Noia, Eugenio Di Sciascio, Azzurra Ragone, and Joseph Trotta. 2019. The importance of being dissimilar in recommendation. In *SAC*. 816–821.
- [10] Eytan Bakshy, Solomon Messing, and Lada A Adamic. 2015. Exposure to ideologically diverse news and opinion on Facebook. *Science* 348, 6239 (2015), 1130–1132.
- [11] Prithu Banerjee, Wei Chen, and Laks V. S. Lakshmanan. 2023. Mitigating Filter Bubbles Under a Competitive Diffusion Model. *Proc. ACM Manag. Data* 1, 2 (2023), 1–26.
- [12] Rianne van den Berg, Thomas N Kipf, and Max Welling. 2017. Graph convolutional matrix completion. *arXiv preprint arXiv:1706.02263* (2017).
- [13] Jaime Carbonell and Jade Goldstein. 1998. The use of MMR, diversity-based reranking for reordering documents and producing summaries. In *SIGIR*. 335–336.
- [14] Dawei Chen, Cheng Soon Ong, and Lexing Xie. 2016. Learning points and routes to recommend trajectories. In *CIKM*. 2227–2232.
- [15] Laming Chen, Guoxin Zhang, and Eric Zhou. 2018. Fast greedy map inference for determinantal point process to improve recommendation diversity. *NeurIPS* 31 (2018).
- [16] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. 2020. SimCLR: A simple framework for contrastive learning of visual representations. In *ICLR*, Vol. 2.
- [17] Xu Chen, Yongfeng Zhang, and Zheng Qin. 2019. Dynamic explainable recommendation based on neural attentive models. In *AAAI*, Vol. 33. 53–60.
- [18] Heng-Tze Cheng, Levent Koc, Jeremiah Harmsen, Tal Shaked, Tushar Chandra, Hrishu Aradhye, Glen Anderson, Greg Corrado, Wei Chai, Mustafa Ispir, Rohan Anil, Zakaria Haque, Lichan Hong, Vihan Jain, Xiaobing Liu, and Hemal Shah. 2016. Wide & deep learning for recommender systems. In *DLRS*. 7–10.
- [19] Wei-Ta Chu and Ya-Lun Tsai. 2017. A hybrid recommendation system considering visual information for predicting favorite restaurants. *WWW* 20, 6 (2017), 1313–1331.
- [20] Aaron Clauset, Cosma Rohilla Shalizi, and Mark EJ Newman. 2009. Power-law distributions in empirical data. *SIAM Rev.* 51, 4 (2009), 661–703.
- [21] Elanor Colleoni, Alessandro Rozza, and Adam Arvidsson. 2014. Echo chamber or public sphere? Predicting political orientation and measuring political homophily in Twitter using big data. *Journal of communication* 64, 2 (2014), 317–332.
- [22] Ting Deng and Wenfei Fan. 2014. On the Complexity of Query Result Diversification. *ACM Trans. Database Syst.* 39, 2 (2014), 15:1–15:46.
- [23] Ting Deng, Wenfei Fan, and Floris Geerts. 2013. On the Complexity of Package Recommendation Problems. *SIAM J. Comput.* 42, 5 (2013), 1940–1986.
- [24] Tommaso Di Noia, Vito Claudio Ostuni, Jessica Rosati, Paolo Tomeo, and Eugenio Di Sciascio. 2014. An analysis of users' propensity toward diversity in recommendations. In *RecSys*. 285–288.
- [25] Patrik Dokoupil, Ludovico Boratto, and Ladislav Peska. 2025. How Do Users Perceive Recommender Systems' Objectives?. In *RecSys*. 165–176.
- [26] Gintare Karolina Dziugaite and Daniel M Roy. 2015. Neural network matrix factorization. *arXiv preprint arXiv:1511.06443* (2015).
- [27] Lihang Fan, Wenfei Fan, Ping Lu, Chao Tian, Muyang Liu, and Qiang Yin. 2023. Enriching Recommendation Models with Logic Conditions. *Proc. ACM Manag. Data* 1, 3 (2023), 210:1–210:28.
- [28] Wenfei Fan, Lihang Fan, Dandan Lin, and Min Xie. 2024. Explaining GNN-based recommendations in logic. *PVLDB* 18, 3 (2024), 715–728.
- [29] Tom Fawcett. 2005. ROC analysis in pattern recognition. *Pattern Recognition Letters* 8 (2005), 861–874.
- [30] Michael Garey and David Johnson. 1979. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman and Company.
- [31] Kiran Garimella, Gianmarco De Francisci Morales, Aristides Gionis, and Michael Mathioudakis. 2017. Reducing controversy by connecting opposing views. In *WSDM*. 81–90.
- [32] Kiran Garimella, Aristides Gionis, Nikos Parotsidis, and Nikolaj Tatti. 2017. Balancing information exposure in social networks. *NeurIPS* 30 (2017).
- [33] Yingqiang Ge, Shuya Zhao, Honglu Zhou, Changhua Pei, Fei Sun, Wenwu Ou, and Yongfeng Zhang. 2020. Understanding echo chambers in e-commerce recommender systems. In *SIGIR*. 2261–2270.
- [34] Quentin Grossetti, Cédric Du Mouza, and Nicolas Travers. 2019. Community-based recommendations on Twitter: Avoiding the filter bubble. In *International Conference on Web Information Systems Engineering*. Springer, 212–227.
- [35] Huifeng Guo, TANG Ruiming, Yunming Ye, Zhenguo Li, and Xiuqiang He. 2017. DeepFM: A Factorization-Machine based Neural Network for CTR Prediction. In *IJCAI*. 1725–1731.
- [36] Myoungji Han, Hyunjoon Kim, Geonmo Gu, Kunsoo Park, and Wook-Shin Han. 2019. Efficient subgraph matching: Harmonizing dynamic programming, adaptive matching order, and failing set together. In *Proc. ACM Manag. Data*. 1429–1446.
- [37] Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, Yongdong Zhang, and Meng Wang. 2020. LightGCN: Simplifying and powering graph convolution network for recommendation. In *SIGIR*. ACM, 639–648.
- [38] Xiangnan He, Xiaoyu Du, Xiang Wang, Feng Tian, Jinhui Tang, and Tat-Seng Chua. 2018. Outer Product-based Neural Collaborative Filtering. In *IJCAI*. 2227–2233.
- [39] Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. 2017. Neural collaborative filtering. In *WWW*. 173–182.
- [40] Cheng-Kang Hsieh, Longqi Yang, Yin Cui, Tsung-Yi Lin, Serge Belongie, and Deborah Estrin. 2017. Collaborative metric learning. In *WWW*. 193–201.
- [41] Shixun Huang, Wenqing Lin, Zhifeng Bao, and Jiachen Sun. 2022. Influence Maximization in Real-World Closed Social Networks. *PVLDB* 16, 2 (2022), 180–192.
- [42] Xing Huang, Dandan Lin, Weiye Huang, Shijie Sun, Jie Wen, and Chuan Chen. 2024. PlatoD2GL: An efficient dynamic deep graph learning system for graph neural network training on billion-scale graphs. In *ICDE*. IEEE, 2421–2434.
- [43] Yanhua Huang, Weikun Wang, Lei Zhang, and Ruiwen Xu. 2021. Sliding spectrum decomposition for diversified recommendation. In *SIGKDD*. 3041–3049.
- [44] Kalervo Järvelin and Jaana Kekäläinen. 2002. Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems (TOIS)* 20, 4 (2002), 422–446.
- [45] Minhao Jiang, Ada Wai-Chee Fu, and Raymond Chi-Wing Wong. 2017. READS: A random walk approach for efficient and accurate dynamic SimRank. *PVLDB* 10, 9 (2017), 937–948.
- [46] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. 2017. LightGBM: A highly efficient gradient boosting decision tree. In *NeurIPS*, Vol. 30. 3146–3154.
- [47] John Paul Kelly and Derek Bridge. 2006. Enhancing the diversity of conversational collaborative recommendations: a comparison. *Artificial Intelligence Review* 25, 1 (2006), 79–95.
- [48] Yehuda Koren. 2008. Factorization meets the neighborhood: A multifaceted collaborative filtering model. In *SIGKDD*. 426–434.
- [49] Yehuda Koren, Robert Bell, and Chris Volinsky. 2009. Matrix factorization techniques for recommender systems. *Computer* 42, 8 (2009), 30–37.
- [50] Ken Lang. 1995. NewsWeeder: Learning to filter netnews. In *Machine learning proceedings 1995*. Elsevier, 331–339.
- [51] Jianxun Lian, Xiaohuan Zhou, Fuzheng Zhang, Zhongxia Chen, Xing Xie, and Guangzhong Sun. 2018. xDeepFM: Combining explicit and implicit feature interactions for recommender systems. In *SIGKDD*. 1754–1763.
- [52] Qingzi Vera Liao and Wai-Tat Fu. 2014. Can you hear me now?: Mitigating the echo chamber effect by source position indicators. In *Computer Supported Cooperative Work (CSCW)*. ACM, 184–196.
- [53] Qingzi Vera Liao and Wai-Tat Fu. 2014. Expert voices in echo chambers: Effects of source expertise indicators on exposure to diverse opinions. In *CHI Conference on Human Factors in Computing Systems*. ACM, 2745–2754.
- [54] Dandan Lin, Shijie Sun, Jingtao Ding, Xuehan Ke, Hao Gu, Xing Huang, Chong-gang Song, Xuri Zhang, Lingling Yi, Jie Wen, and Chuan Chen. 2022. PlatoGL: Effective and scalable deep graph learning system for graph-enhanced real-time recommendation. In *CIKM*. 3302–3311.
- [55] Dandan Lin, Raymond Chi-Wing Wong, Min Xie, and Victor Junqiu Wei. 2020. Index-free approach with theoretical guarantee for efficient random walk with restart query. In *ICDE*. IEEE, 913–924.
- [56] Greg Linden, Brent Smith, and Jeremy York. 2003. Amazon.com recommendations: Item-to-item collaborative filtering. *IEEE Internet Computing* 7, 1 (2003), 76–80.
- [57] Weiwen Liu, Yunjia Xi, Jiarui Qin, Xinyi Dai, Ruiming Tang, Shuai Li, Weinan Zhang, and Rui Zhang. 2023. Personalized Diversification for Neural Re-ranking in Recommendation. In *ICDE*. IEEE, 802–815.

- [58] Beth Logan and Ariel Salomon. 2001. A content-based music similarity function. *Cambridge Research Labs-Tech Report* (2001).
- [59] Xin Luo, Mengchu Zhou, Yunni Xia, and Qingsheng Zhu. 2014. An efficient non-negative matrix-factorization-based approach to collaborative filtering for recommender systems. *IEEE Trans. Industr. Inform.* 10, 2 (2014), 1273–1284.
- [60] Stephann Makri, Ann Blandford, Mel Woods, Sarah Sharples, and Deborah Maxwell. 2014. “Making my own luck”: Serendipity strategies and how to support them in digital information environments. *J. Assoc. Inf. Sci. Technol.* 65, 11 (2014), 2179–2194.
- [61] Antonis Matakos, Çigdem Aslay, Esther Galbrun, and Aristides Gionis. 2022. Maximizing the Diversity of Exposure in a Social Network. *IEEE Trans. Knowl. Data Eng.* 34, 9 (2022), 4357–4370.
- [62] Dana McKay, Stephann Makri, Shanton Chang, and George Buchanan. 2020. On Birthing Dancing Stars: The Need for Bounded Chaos in Information Interaction. In *SIGIR Conference on Human Information Interaction and Retrieval (CHIIR)*. ACM, 292–302.
- [63] Lien Michiels, Jens Leysen, Annelien Smets, and Bart Goethals. 2022. What are filter bubbles really? A review of the conceptual and empirical work. In *UMAP*. 274–279.
- [64] Liang Pang, Jun Xu, Qingyao Ai, Yanyan Lan, Xueqi Cheng, and Jirong Wen. 2020. SetRank: Learning a permutation-invariant ranking model for information retrieval. In *SIGIR*. 499–508.
- [65] Eli Pariser. 2011. *The filter bubble: What the Internet is hiding from you*. Penguin UK.
- [66] Hao Qian, Qintong Wu, Kai Zhang, Zhiqiang Zhang, Lihong Gu, Xiaodong Zeng, Jun Zhou, and Jinjie Gu. 2022. Scope-aware re-ranking with gated attention in feed. In *WSDM*. 804–812.
- [67] Xubo Qin, Zhicheng Dou, and Ji-Rong Wen. 2020. Diversifying search results using self-attention network. In *CIKM*. 1265–1274.
- [68] Badrul Sarwar, George Karypis, Joseph Konstan, and John Riedl. 2001. Item-based collaborative filtering recommendation algorithms. In *WWW*. 285–295.
- [69] Suvash Sedhain, Aditya Krishna Menon, Scott Sanner, and Lexing Xie. 2015. AutoRec: Autoencoders meet collaborative filtering. In *WWW*. 111–112.
- [70] Yingdan Shang, Bin Zhou, Xiang Zeng, Ye Wang, Han Yu, and Zhong Zhang. 2022. Predicting the popularity of online content by modeling the social influence and homophily features. *Frontiers in Physics* 10 (2022), 915756.
- [71] Upendra Shardanand and Pattie Maes. 1995. Social information filtering: Algorithms for automating “word of mouth”. In *the SIGCHI conference on Human factors in computing systems*. 210–217.
- [72] Xiaoyu Shi, Quanliang Liu, Hong Xie, Di Wu, Bo Peng, MingSheng Shang, and Defu Lian. 2023. Relieving popularity bias in interactive recommendation: A diversity-novelty-aware reinforcement learning approach. *ACM Transactions on Information Systems* 42, 2 (2023), 1–30.
- [73] Florian Strub, Romaric Gaudel, and Jérémie Mary. 2016. Hybrid recommender system based on autoencoders. In *DLRS*. 11–16.
- [74] Florian Strub and Jeremie Mary. 2015. Collaborative filtering with stacked denoising autoencoders and sparse inputs. In *NIPS workshop on machine learning for eCommerce*.
- [75] Huifeng Sun, Yong Peng, Junliang Chen, Chuanchang Liu, and Yuzhuo Sun. 2011. A New Similarity Measure Based on Adjusted Euclidean Distance for Memory-based Collaborative Filtering. *JSW* 6, 6 (2011), 993–1000.
- [76] Zhu Sun, Qing Guo, Jie Yang, Hui Fang, Guibing Guo, Jie Zhang, and Robin Burke. 2019. Research commentary on recommendations with side information: A survey and research directions. *Electronic Commerce Research and Applications* 37 (2019).
- [77] Cass R. Sunstein. 2017. *Republic: Divided Democracy in the Age of Social Media*. Princeton University Press.
- [78] Jiayi Tang and Ke Wang. 2018. Personalized top-n sequential recommendation via convolutional sequence embedding. In *WSDM*. 565–573.
- [79] Sijing Tu, Çigdem Aslay, and Aristides Gionis. 2020. Co-exposure Maximization in Online Social Networks. In *NeurIPS*.
- [80] Farman Ullah, Ghulam Sarwar, Sung Chang Lee, Yun Kyung Park, Kyeong Deok Moon, and Jin Tae Kim. 2012. Hybrid recommender system with temporal information. In *ICOLN*. IEEE, 421–425.
- [81] Aaron Van den Oord, Sander Dieleman, and Benjamin Schrauwen. 2013. Deep content-based music recommendation. *NeurIPS* 26 (2013).
- [82] Saúl Vargas, Linas Baltrunas, Alexandros Karatzoglou, and Pablo Castells. 2014. Coverage, redundancy and size-awareness in genre diversity for recommender systems. In *RecSys*. 209–216.
- [83] Soroush Vosoughi, Deb Roy, and Sinan Aral. 2018. The spread of true and false news online. *Science* 359, 6380 (2018), 1146–1151.
- [84] Xiang Wang, Xiangnan He, Yixin Cao, Meng Liu, and Tat-Seng Chua. 2019. KGAT: Knowledge Graph Attention Network for Recommendation. In *SIGKDD*. ACM, 950–958.
- [85] Xiang Wang, Xiangnan He, Meng Wang, Fuli Feng, and Tat-Seng Chua. 2019. Neural graph collaborative filtering. In *SIGIR*. ACM, 165–174.
- [86] Xinxi Wang and Ye Wang. 2014. Improving content-based and hybrid music recommendation using deep learning. In *ACM Multimedia*. 627–636.
- [87] Zihong Wang, Yingxia Shao, Jiyuan He, Jinbao Liu, Shitao Xiao, Tao Feng, and Ming Liu. 2023. Diversity-aware deep ranking network for recommendation. In *CIKM*. 2564–2573.
- [88] Qiong Wu, Yong Liu, Chunyan Miao, Binqiang Zhao, Yin Zhao, and Lu Guan. 2019. PD-GAN: Adversarial learning for personalized diversity-promoting recommendation. In *IJCAI*. Vol. 19. 3870–3876.
- [89] Minhui Xie, Kai Ren, Youyou Lu, Guangxu Yang, Qingxing Xu, Bihai Wu, Jiazhen Lin, Hongbo Ao, Wanhong Xu, and Jiwu Shu. 2020. Kraken: Memory-efficient continual learning for large-scale real-time recommendations. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–17.
- [90] Eric Xing, Michael Jordan, Stuart J Russell, and Andrew Ng. 2002. Distance metric learning with application to clustering with side-information. *NeurIPS* 15 (2002).
- [91] Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L Hamilton, and Jure Leskovec. 2018. Graph convolutional neural networks for Web-scale recommender systems. In *SIGKDD*. 974–983.
- [92] Junliang Yu, Hongzhi Yin, Xin Xia, Tong Chen, Jundong Li, and Zi Huang. 2023. Self-supervised learning for recommender systems: A survey. *IEEE Trans. Knowl. Data Eng.* 36, 1 (2023), 335–355.
- [93] Min Zhang, Dongxin Zhang, Yin Zhang, Kristin Yeager, and Taylor N Fields. 2023. An exploratory study of Twitter metrics for measuring user influence. *Journal of Informetrics* 17, 4 (2023), 101454.
- [94] Yuying Zhao, Yu Wang, Yunchao Liu, Xueqi Cheng, Charu C Aggarwal, and Tyler Derr. 2025. Fairness and diversity in recommender systems: A survey. *ACM Transactions on Intelligent Systems and Technology* 16, 1 (2025), 1–28.
- [95] Lei Zheng, Vahid Noroozi, and Philip S Yu. 2017. Joint deep modeling of users and items using reviews for recommendation. In *WSDM*. 425–434.
- [96] Ziwei Zhu, Yun He, Xing Zhao, and James Caverlee. 2021. Popularity bias in dynamic recommendation. In *SIGKDD*. 2439–2449.
- [97] Cai-Nicolas Ziegler, Georg Lausen, and Lars Schmidt-Thieme. 2004. Taxonomy-driven computation of product recommendations. In *CIKM*. 406–415.

Input: graph G , a set Σ_{TIE} of TIEs, $\mathcal{M}_{\text{rec}}$, $\mathcal{M}_{\text{dis}}$, $\mathcal{M}_{\text{aggr}}$, and $\text{Req}(r_1, r_2, \alpha)$
Output: a set Σ_{OMR} of OMRs

1. $\Sigma_{\text{OMR}} := \emptyset$; $\mathcal{S} := \text{buildSchema}(G)$;
2. **for** each TIE $\varphi_T \in \Sigma_{\text{TIE}}$ **do**
3. $Q := \text{GenPatterns}(Q_T, \mathcal{S}, \alpha)$;
4. **for** each $Q \in Q$ **do**
5. $\Theta_{\text{dis}}, \Theta_{\text{aggr}} := \text{SelectThresholds}(Q, \varphi_T, G)$;
6. $\Delta := \text{GenPredicates}(Q, X_T, \Theta_{\text{dis}}, \Theta_{\text{aggr}})$;
7. **for** each $(\delta_d, \delta_a) \in \Theta_{\text{dis}} \times \Theta_{\text{aggr}}$ **do**
8. $X := X_T \cup \{\text{Rec}(x_0, y_0), \mathcal{M}_{\text{dis}}(y_0, w_0) \geq \delta_d, \mathcal{M}_{\text{aggr}}(w_0) \geq \delta_a\}$;
9. $\varphi := Q[x_0, y_0, w_0](X \rightarrow \text{Rec}_d(x_0, w_0))$;
10. **if** φ is r_1 -frequent and r_2 -confident **then**
11. $\Sigma_{\text{OMR}} := \Sigma_{\text{OMR}} \cup \{\varphi\}$;
12. **elseif** φ is r_1 -frequent but not r_2 -confident **then**
13. $\Sigma'_{\text{OMR}} := \text{Expand}(G, Q, X, \Delta)$; $\Sigma_{\text{OMR}} := \Sigma_{\text{OMR}} \cup \Sigma'_{\text{OMR}}$;
14. **Return** Σ_{OMR} ;

Figure 11: OMRLearner.

APPENDIX

This appendix is organized as follows.

- Appendix A: Details about Rule Discovery Algorithm
- Appendix B: Details of CTR Algorithm
- Appendix C: Details of TopK Algorithm
- Appendix D: Details about Experiments

Appendix A: Details about Rule Discovery Algorithm.

This section presents the details of rule discovery algorithm OMRLearner, which mines OMRs by extending the learning-based TIE discovery algorithm [27]. We start with the criteria.

Criteria. We use *support* and *confidence* to measure the quality of OMRs. Given an OMR $\varphi = Q[x_0, y_0, w_0](X \rightarrow \text{Rec}_d(x_0, w_0))$ and a graph G , denote by $\text{Rec}_d(G, \varphi)$ the set of all alternatives predicted by φ . That is, $(u, w) \in \text{Rec}_d(G, \varphi)$ if (a) $u = h(x_0)$, $v = h(y_0)$ and $w = h(w_0)$ for some match h of Q in G , and (b) $h \models X$.

Support. The support of an OMR φ measures how often φ can be applied. It is assessed *w.r.t.* training data $\mathcal{T}$. Here $\mathcal{T}$ is a set of user-item pairs within a graph G , which are real-life interactions and can serve as ground truth. Then the support of φ over a graph G *w.r.t.* $\mathcal{T}$, denoted by $\text{supp}(\varphi, G, \mathcal{T})$, is defined as

$$\text{supp}(\varphi, G, \mathcal{T}) = |\text{Rec}_d(G, \varphi) \cap \{(u, w) \mid (u, \text{clicked}, w) \in \mathcal{T}\}|.$$

This metric is *anti-monotonic* *w.r.t.* a partial order $\preceq$ on the domain of OMRs. Given two OMRs $\varphi(Q, X)$ and $\varphi'(Q', X')$, $\varphi \preceq \varphi'$ if (a) Q is a subgraph of Q' and (b) all the predicates in X are also included in X' . One can verify that $\text{supp}(\varphi', G, \mathcal{T}) \leq \text{supp}(\varphi, G, \mathcal{T})$ if $\varphi \preceq \varphi'$.

Confidence. Given an OMR φ , its *confidence* over a graph G *w.r.t.* training data $\mathcal{T}$, denoted as $\text{conf}(\varphi, G, \mathcal{T})$, is defined as

$$\text{conf}(\varphi, G, \mathcal{T}) = \frac{\text{supp}(\varphi, G, \mathcal{T})}{|\text{Rec}_d(G, \varphi) \cap \{(u, w) \mid (u, \tau, w) \in \mathcal{T}\}|}$$

where τ is clicked or non-clicked. The confidence measures the association strength between the precondition of φ and the predictions made by φ , *i.e.*, the “precision” of φ by referencing ground truth in $\mathcal{T}$. An OMR with high confidence helps us increase the diversity.

OMR discovery. We now formalize the OMR discovery problem.

- *Input:* A graph G , the training data $\mathcal{T}$ in G , a set Σ_{TIE} of TIEs, $\mathcal{M}_{\text{rec}}$, $\mathcal{M}_{\text{dis}}$, $\mathcal{M}_{\text{aggr}}$, a support threshold r_1 , a confidence thresh-

old r_2 , and a positive integer α .

- *Output:* A set Σ_{OMR} of OMRs such that for each $\varphi \in \Sigma_{\text{OMR}}$, $\text{supp}(\varphi, G, \mathcal{T}) \geq r_1$, $\text{conf}(\varphi, G, \mathcal{T}) \geq r_2$, and the pattern of φ has at most α variables.

We use $\text{Req}(r_1, r_2, \alpha)$ to represent an OMR discovery requirement. The problem aims to find a set of OMRs that conform to $\text{Req}(r_1, r_2, \alpha)$, where α controls the cost of the discovery process.

Intuition. To avoid mining OMRs from scratch, we leverage pre-mined TIEs as the basis for OMR discovery. Specifically, given a pre-mined TIE $\varphi_T(Q_T, X_T)$ in Σ_{TIE} , OMRLearner derives a set of candidate patterns for OMRs by preserving the pattern Q_T of φ_T at the pivot variables (x_0, y_0) and extending it via (a) a *bridge path* to an alternative variable w_0 and (b) a *star pattern* centered at w_0 .

However, enumerating such extensions over the entire graph is computationally expensive, since it requires exploring a large number of instance-level paths and star expansions around each match of pivots (x_0, y_0) , and many of which are structurally redundant and differ only in their concrete instances. To reduce the search space, OMRLearner performs the enumeration over a *graph schema*, *i.e.*, a compact type-level abstraction of the graph that merges structurally identical instance-level extensions.

In addition, unlike TIEs, the precondition of an OMR incorporates dissimilarity and popularity constraints, captured by $\mathcal{M}_{\text{dis}}$ and $\mathcal{M}_{\text{aggr}}$, respectively. Instead of assuming these thresholds are user-specified or exhaustively searching over them, OMRLearner learns them by extracting quantile-based threshold candidates from the empirical distributions of $\mathcal{M}_{\text{dis}}$ and $\mathcal{M}_{\text{aggr}}$ over the raw matches of candidate patterns for OMRs. During predicate generation, it further exploits the anti-monotonicity of support to prune candidates whose support already falls below the given threshold, thus avoiding exhaustive exploration of predicate combinations.

Overview. Given a set Σ_{TIE} of pre-mined TIEs, OMRLearner processes each TIE φ_T in Σ_{TIE} to derive OMRs in two steps.

- (1) *Pattern generation.* Starting from the TIE pattern Q_T of φ_T , OMRLearner derives a set Q of candidate OMR patterns by preserving Q_T at the pivot variables (x_0, y_0) and extending it with (a) a bridge path ρ_B connecting the anchor variable y_0 to a new alternative variable w_0 of item type, and (b) a star pattern Q_w centered at w_0 , *i.e.*, $Q = \{Q^{(i)}\}$, where $Q^{(i)} = Q_T \cup \rho_B^{(i)} \cup Q_w^{(i)}$. To do this, it enumerates those extensions over the graph schema $\mathcal{S}$, rather than directly over the input graph.
- (2) *Predicate generation.* For each candidate pattern in Q , it constructs a set of preconditions by extending X_T of the TIE φ_T with dissimilarity and popularity constraints, whose thresholds are selected using a *quantile-based selection* strategy. It then evaluates support and confidence, and outputs the set Σ_{OMR} of candidate OMRs satisfying support and confidence thresholds.

Algorithm. As outlined in Figure 11, OMRLearner takes as input a graph G , a set Σ_{TIE} of pre-mined TIEs, models $\mathcal{M}_{\text{rec}}$, $\mathcal{M}_{\text{dis}}$, $\mathcal{M}_{\text{aggr}}$, and $\text{Req}(r_1, r_2, \alpha)$. It outputs a set Σ_{OMR} of discovered OMRs.

The algorithm first initializes Σ_{OMR} to the empty set and constructs the graph schema $\mathcal{S}$ from G via procedure `buildSchema` (Line 1). It then processes each TIE $\varphi_T \in \Sigma_{\text{TIE}}$ one by one (Lines 2-13). For the current TIE φ_T , it generates a set Q of candidate OMR

patterns from the TIE pattern Q_T via procedure GenPatterns, guided by the schema S and bounded by α (Line 3). For each candidate pattern $Q \in \mathcal{Q}$, it selects two sets of threshold candidates, Θ_{dis} and Θ_{aggr} , for the dissimilarity and popularity measure, respectively, via procedure SelectThresholds (Line 5). It then invokes procedure GenPredicates to generate a set Δ of candidate predicates for the current pattern Q , to be used in subsequent rule expansion (Line 6).

Next, it starts mining preconditions for Q by enumerating all threshold pairs $(\delta_d, \delta_a) \in \Theta_{\text{dis}} \times \Theta_{\text{aggr}}$ (Lines 7-13). Specifically, for each pair (δ_d, δ_a) , it first constructs a base precondition X by extending the TIE precondition X_T with the recommendation predicate $\text{Rec}(x_0, y_0)$, the dissimilarity constraint $\mathcal{M}_{\text{dis}}(y_0, w_0) \geq \delta_d$, and the popularity constraint $\mathcal{M}_{\text{aggr}}(w_0) \geq \delta_a$ (Line 8). Based on current Q and X , it forms a candidate OMR φ (Line 9). If φ is both r_1 -frequent, *i.e.*, its support is no smaller than the support threshold r_1 , and r_2 -confident, *i.e.*, its confidence is greater than the confidence threshold r_2 , the algorithm adds φ to Σ_{OMR} (Lines 10-11). Otherwise, if φ is r_1 -frequent but not r_2 -confident, the algorithm refines it via procedure Expand, which appends predicates from Δ in a level-wise manner to derive more specific OMRs satisfying the support and confidence requirements. All expanded OMRs returned by this procedure are then added to Σ_{OMR} (Lines 12-13). After all TIEs have been explored, the algorithm returns the final set Σ_{OMR} (Line 14).

Below we present the details of procedures used in OMR Learner.

Procedure buildSchema. Given a data graph $G(V, E)$, procedure buildSchema constructs its graph schema $S(V_S, E_S)$, where each node $A \in V_S$ represents a vertex type occurring in G , and each typed edge $(A, l, B) \in E_S$ indicates that there exists at least one edge $(v, l, v') \in E$ in G such that $L(v) = A$ and $L(v') = B$.

To build the schema, this procedure scans all vertices and edges in G and identifies the set of vertex types appearing in G . For each vertex type A , it creates a schema node in S . Then, for each edge $(u, l, v) \in E$, where $L(u) = A$ and $L(v) = B$, it inserts a schema edge (A, l, B) into E_S . Duplicate schema edges are merged.

Procedure GenPatterns. Given a TIE pattern Q_T , graph schema S and a size bound α , this procedure generates a set $\mathcal{Q}$ of candidate OMR patterns by extending Q_T with a bridge path and an alternative-centered star pattern.

It first generates *bridge paths* from the schema S . Specifically, starting from the type of the anchor variable y_0 in Q_T , it enumerates schema paths of length at most L that end at the item type, so that the endpoint can be instantiated as the alternative item variable w_0 . Each schema path induces a bridge path ρ_B by introducing fresh variables for intermediate schema nodes. For star pattern Q_w , it considers all schema edges incident to the type of w_0 , and treats each as a branch rooted at w_0 . By enumerating those branches, it generates a set of star pattern Q_w pivoted at w_0 describing the local neighborhood of the alternative variable w_0 .

For each pair (ρ_B, Q_w) , it forms a candidate OMR pattern Q as $Q_T \cup \rho_B \cup Q_w$. It prunes those candidates whose size exceed α .

Procedure SelectThresholds. Given a candidate pattern Q , a TIE φ_T and a graph G , this procedure selects two sets Θ_{dis} and Θ_{aggr} of candidate thresholds for $\mathcal{M}_{\text{dis}}$ and $\mathcal{M}_{\text{aggr}}$, respectively.

To do this, it first enumerates the matches of Q in G that satisfy the TIE precondition X_T . For each match h , it computes the pair of scores by $\mathcal{M}_{\text{dis}}$ and $\mathcal{M}_{\text{aggr}}$, *i.e.*, $d(h) = \mathcal{M}_{\text{dis}}(h(y_0), h(w_0))$ and

$a(h) = \mathcal{M}_{\text{aggr}}(h(w_0))$. This yields two empirical distributions, one for dissimilarity and the other for the popularity.

Since the observed values in these distributions may be numerous, directly using all distinct values as thresholds would lead to an unnecessarily large search space. Thus, this procedure applies quantile-based strategy. That is, it sorts each distribution, extracts cutoffs as predefined quantile levels, removes duplicates, and returns the resulting ordered sets Θ_{dis} and Θ_{aggr} .

Procedure GenPredicates. Given a candidate pattern Q , the TIE precondition X_T , and the threshold candidate sets Θ_{dis} and Θ_{aggr} , this procedure generates a set Δ of candidate predicates.

It first instantiates numeric predicates associated with the alternative item, and then derives additional predicates from the newly introduced part of Q , *i.e.*, the bridge path ρ_B and star pattern Q_w , by using the predicate construction method in [27]. To control the search space, this procedure removes predicates already implied by the TIE precondition X_T , inconsistent with the schema of Q , or redundant under equivalent variable roles.

Procedure Expand. Given G , a candidate pattern Q , a base precondition X , and a predicate set Δ , this procedure returns a set Σ'_{OMR} of OMRs that satisfies the support and confidence thresholds.

Specifically, it performs a level-wise manner over refinements of the form of $X' = X \cup \Gamma$ where $\Gamma \subseteq \Delta$. For each refinement, it evaluates the corresponding OMR against the support and confidence thresholds. A rule is returned if it satisfies both thresholds. A candidate violating the support threshold is pruned by anti-monotonicity. The procedure terminates when no promising refinements remain.

Level-wise manner. The expansion process starts with $X_0 = \{X\}$, where X includes ML predicates $\mathcal{M}_{\text{rec}}(x_0, y_0)$, $\mathcal{M}_{\text{dis}}(y_0, w_0) \geq \delta_d$ and $\mathcal{M}_{\text{aggr}}(w_0) \geq \delta_a$ as in Figure 11.

For each expansion to generate X_i , it picks a predicate p from Δ and extends X to $X \cup p$; it checks the derived OMR $\varphi = Q[x_0, y_0, w_0](X \cup p \rightarrow \text{Rec}_d(x_0, w_0))$ w.r.t. $\text{Req}(r_1, r_2, \alpha)$. If $\text{supp}(\varphi, G, \mathcal{T}) \geq r_1$ and $\text{conf}(\varphi, G, \mathcal{T}) \geq r_2$, then it adds $X \cup p$ to $\mathcal{X}$. Otherwise if only $\text{supp}(\varphi, G, \mathcal{T}) \geq r_1$ holds, then it adds $X \cup p$ to X_{i+1} for further exploration. Note that $X \cup p$ is not picked if $\text{supp}(\varphi, G, \mathcal{T}) < r_1$, due to the anti-monotonicity of the support. When no X_i can be further extended, the process terminates.

Appendix B: Details of CTR Algorithm

This section presents more details of the CTR algorithm, including the ranking score and rooted DAG. In addition, it provides the proofs of lemmas and theorems in Section 4.

Appendix B-1: Ranking Score. Given an OMR φ , the contribution $\Delta_\varphi(u, v, w)$ of dissimilarity given by φ is defined as:

$$\Delta_\varphi(u, v, w) = \sum_{z \in \bar{x} \cup \bar{y}} s(h(z)) = \sum_{z \in \bar{x} \cup \bar{y}} \sum_{A \text{ in } X} \mathbb{I}_{z.A} \cdot \beta_{h(z).A},$$

where $z \in \bar{x} \cup \bar{y}$ is a variable in pattern Q of φ , $h(z)$ is a matched vertex at z by a match h of Q such that $h \models X$, $s(h(z))$ is the score of $h(z)$, $\mathbb{I}_{z.A}$ is a Boolean function that checks whether $z.A$ appears in X of φ , and $\beta_{h(z).A}$ is the feature (*i.e.*, attribute value) importance of $h(z).A$. Intuitively, $\beta_{h(z).A}$ distinguishes the priority of actual attribute values in different h , *e.g.*, a poster with a larger number of fans is more desirable (*i.e.*, an opinion leader in social networks).

Appendix B-2: Rooted DAG. We construct a rooted DAG Q_D^H

from Q^H by treating the user variable x_0 as the root. We perform a BFS traversal from x_0 and orient edges from higher to lower levels, ensuring that Q_D^H is acyclic and that only x_0 has no incoming edges.

To improve pruning efficiency, variables at the same level are ordered by selectivity. Specifically, we group variables by label and sort the groups in ascending order of their frequency in G ; within each group, variables are ordered by decreasing degree.

Appendix B-3: Procedure InitCand. Given user u and rule group H , this procedure initializes a shared candidate map

$$C^H = \{C^H(z) \subseteq V \mid z \in \text{Var}(Q^H)\}.$$

It sets $C^H(x_0) = \{u\}$ and, for each $z \neq x_0$, defines

$$C^H(z) = \{v \in V \mid v \text{ satisfies the unary constraints of } z\}.$$

A vertex v is included in $C^H(z)$ if $L_G(v) = L_{Q^H}(z)$, $\deg_G(v) \geq \deg_{Q^H}(z)$, and moreover, v satisfies all constant predicates on z in X^H (e.g., $z.A \oplus c$, ML predicates such as $M_{\text{rec}}(x_0, y_0) \otimes \delta$, $M_{\text{dis}}(y_0, w_0) \otimes \delta$, and popularity predicates $M_{\text{aggr}}(w_0) \otimes \delta$).

Appendix B-4: Proof of Lemma 1

Proof: Consider that a set Σ^w of OMRs in Σ that are applicable at w . Note that these OMRs are processed subsequently in our CTR algorithm. For the first time that w is discovered by an OMR φ in Σ^w , i.e., $w \notin A_\varphi$, its score is computed as below:

$$s_1 = (1 - \lambda) \cdot s_{\text{acc}}(u, w) + \lambda \cdot \text{sim}_T(v, w) \cdot \frac{1}{|\Sigma|} \cdot s_{\text{dis}}(u, v, w, \varphi).$$

Let s_i denote the addition for i -th OMR φ_i in Σ^w where $i > 1$. We have the following derivations.

$$\begin{aligned} s_1 + \sum_{i=2}^{|\Sigma^w|} s_i &= s_1 + \sum_{i=2}^{|\Sigma^w|} \lambda \cdot \text{sim}_T(v, w) \cdot \frac{1}{|\Sigma|} \cdot s_{\text{dis}}(u, v, w, \varphi_i) \\ &= s_1 + \lambda \cdot \frac{1}{|\Sigma|} \cdot \text{sim}_T(v, w) \cdot \sum_{\varphi \in \Sigma^w \setminus \varphi_1} s_{\text{dis}}(u, v, w, \varphi) \\ &= (1 - \lambda) \cdot s_{\text{acc}}(u, w) + \lambda \cdot \frac{1}{|\Sigma|} \cdot \text{sim}_T(v, w) \cdot \sum_{\varphi \in \Sigma^w} s_{\text{dis}}(u, v, w, \varphi) \\ &= s(u, v, w). \end{aligned}$$

This completes the proof. $\square$

Appendix B-5: Proof of Lemma 2

Proof: (1) We first prove that each local refinement operator F_φ is monotone. For a fixed rule $\varphi \in H$, and $C^H \sqsubseteq C'^H$ such that $C^H(z) \subseteq C'^H(z)$ for all z , we prove that $F_\varphi(C^H) \sqsubseteq F_\varphi(C'^H)$. When the context is clear, we use C^H and C interchangeably.

Consider any variable $z \in \text{Var}(Q^H)$ and any vertex $v \in F_\varphi(C)(z)$. If $z \notin \text{Var}(Q_\varphi)$, then by construction of localRefinement, the candidate map remains unchanged, i.e., $F_\varphi(C)(z) = C(z)$ and $F_\varphi(C')(z) = C'(z)$. Since $C(z) \subseteq C'(z)$, we have $v \in F_\varphi(C')(z)$. Now, suppose $z \in \text{Var}(Q_\varphi)$. By the construction of localRefinement, $v \in F_\varphi(C)(z)$ means that v satisfies all constraints enforced by X in φ in the forward and backward passes w.r.t. the candidate map C . Since $C \sqsubseteq C'$, all candidate sets available under C are also available under C' . Thus, each vertex used to justify $v \in F_\varphi(C)(z)$ is still present in the corresponding set under C' . Hence, $v \in F_\varphi(C')(z)$. Since z and v are arbitrary, this proves that $F_\varphi(C) \subseteq F_\varphi(C')$.

(2) We prove that the group refinement operator F_H is also monotone. Since union is monotone w.r.t. the set inclusion and each F_φ is monotone, we have the following:

$$F_H(C) = \bigcup_{\varphi \in H} F_\varphi(C) \sqsubseteq \bigcup_{\varphi \in H} F_\varphi(C') = F_H(C').$$

Thus, F_H is monotone.

(3) We prove that the group refinement can reach a unique fixpoint. First of all, since each local refinement $F_\varphi(C) \subseteq C$, we have, for each $z \in \text{Var}(Q^H)$, $F_H(C)(z) = \bigcup_{\varphi \in H} F_\varphi(C)(z) \subseteq C(z)$. Thus, $F_H(C) \subseteq C$, indicating that F_H is also contractive.

Then, starting from the initial candidate map, denoted by C , we define the updated candidate map as $F_H^{i+1} = F_H^i(C)$. By contractiveness, $F_H^{i+1} \subseteq F_H^i$ for all $i \geq 0$. There exists some $k \geq 0$ such that $F_H^{k+1} = F_H^k$. Let $F_H^\infty = F_H^k$, F_H^∞ is a fixpoint. The proof completes. $\square$

Appendix B-6: Proof of Theorem 3

Proof: (1) We first prove the soundness. Let $(v, w) \in \text{Cpl}(u, H)$. By fixpoint extraction, there exists some $\varphi \in H \setminus \{\varphi_T\}$ and a witness match h_φ satisfying X_φ such that $h_\varphi(x_0) = u$, $h_\varphi(y_0) = v$, and $h_\varphi(w_0) = w$. Since v is output as an anchor, there also exists a witness h_T of φ_T such that $h_T(x_0) = u$ and $h_T(y_0) = v$. Hence, the sequential algorithm first outputs v in the TIE stage and then outputs w for v in the OMR stage, so $(v, w) \in \text{Seq}(u, H)$.

(2) We then prove the completeness. Let $(v, w) \in \text{Seq}(u, H)$. In this case, there exists a TIE witness h_T and an OMR witness h_φ from some $\varphi \in H \setminus \{\varphi_T\}$ such that $h_T(x_0) = h_\varphi(x_0) = u$, $h_T(y_0) = h_\varphi(y_0) = v$, and $h_\varphi(w_0) = w$. Since φ is compatible with φ_T on the pivot variables, $h = h_T \cup h_\varphi$ is well-defined. Each vertex in h belongs to the initialized candidate map and is supported by valid witnesses for the forward/backward passes and moreover, no iteration of group refinements F_H removes it. Thus, $v \in F_H^\infty(y_0)$ and $w \in F_H^\infty(w_0)$, making $(v, w) \in \text{Cpl}(u, H)$.

Combining both, we have $\text{Seq}(u, H) = \text{Cpl}(u, H)$. $\square$

Appendix B-7: Proof of Theorem 4

Proof: To prove this, we show the stronger inclusion $\mathcal{N}_{\text{Cpl}} \subseteq \mathcal{N}_{\text{Seq}}$. For a fixed OMR $\varphi \in H \setminus \{\varphi_T\}$. Let C_H^i and $C_{\varphi, \text{Seq}}^i$ be the candidate maps maintained at the i -th iteration by the coupled and sequential algorithm, respectively. We want to prove that for each $i \geq 0$,

$$C_H^i(y_0) \subseteq C_{\varphi, \text{Seq}}^i(y_0), \text{ and } C_H^i(w_0) \subseteq C_{\varphi, \text{Seq}}^i(w_0).$$

We prove the claim by induction. When $i = 0$, it holds since the coupled algorithm initializes the candidate map by performing on the joint pattern Q^H , which contains all constraints of Q_φ and possibly more, and hence is no weaker than the isolated initialization for φ . Assume that it holds at the i -th iteration, i.e.,

$$C_H^i(y_0) \subseteq C_{\varphi, \text{Seq}}^i(y_0), \text{ and } C_H^i(w_0) \subseteq C_{\varphi, \text{Seq}}^i(w_0).$$

We prove that at the $(i + 1)$ -th iteration,

$$C_H^{i+1}(y_0) \subseteq C_{\varphi, \text{Seq}}^{i+1}(y_0), \text{ and } C_H^{i+1}(w_0) \subseteq C_{\varphi, \text{Seq}}^{i+1}(w_0).$$

For the sequential algorithm that refines the candidate $C_{\varphi, \text{Seq}}^i$ by enforcing the constraints of φ . We denote this refinement by F_φ , and for variable y_0 (or w_0), we have

$$\begin{aligned} C_{\varphi, \text{Seq}}^{i+1}(y_0) &= F_\varphi(C_{\varphi, \text{Seq}}^i(y_0)), \text{ and } C_{\varphi, \text{Seq}}^{i+1}(w_0) = F_\varphi(C_{\varphi, \text{Seq}}^i(w_0)). \\ \text{Since } C_H^i(y_0) &\subseteq C_{\varphi, \text{Seq}}^i(y_0) \text{ and } F_\varphi \text{ is monotone, we get } F_\varphi(C_H^i(y_0)) \\ &\subseteq F_\varphi(C_{\varphi, \text{Seq}}^i(y_0)). \text{ Similarly, } F_\varphi(C_H^i(w_0)) \subseteq F_\varphi(C_{\varphi, \text{Seq}}^i(w_0)). \end{aligned}$$

For our coupled algorithm, it evaluates all rules in H over the shared candidate map. The updated C_H^{i+1} is obtained by combin-

Input: $G, u, \Sigma_{\text{OMR}}, \Sigma_{\text{TIE}}, \mathcal{M}_{\text{rec}}, \mathcal{M}_{\text{dis}}, \mathcal{M}_{\text{agg}}, \lambda, \delta_H$ and k .

Output: a set $K(u)$ of anchor items and a set $K_d(u)$ of alternative items.

1. $\Psi := \emptyset; K(u) := \emptyset; K_d(u) := \emptyset;$
2. $K(u) := \text{TopKAnchor}(u, \Sigma_{\text{TIE}}, \mathcal{M}_{\text{rec}}, \delta_H, k);$
3. $H, Q^H, Q_D^H := \text{constructGroup}(u, K(u), \Sigma_{\text{OMR}});$
4. $C^H := \text{InitCand}(Q^H, u); F_H := C^H;$
5. Initialize best_w and $\hat{s}(u, w)$ for each $w \in F_H(w_0);$
6. **for** each $v \in F_H(y_0)$ **do**
7. **for** each $\varphi \in H$ **do**
8. $\text{UB}_\varphi(u, v) := \hat{y}(v) \cdot \Delta_\varphi^{\max};$
9. **for** each $(v, w) \in F_H(y_0, w_0)$ **do**
10. $\hat{s}(u, v, w) := \lambda \cdot \frac{1}{|H|} \sum_{\varphi \in H} \text{UB}_\varphi(u, v) + (1 - \lambda) \cdot s_{\text{acc}}(u, w);$
11. **if** $\hat{s}(u, v, w) \geq \hat{s}(u, w)$ **then** $\hat{s}(u, w) := \hat{s}(u, v, w);$
12. **while** F_H does not converge **do**
13. **for** each $\varphi \in H$ **do**
14. $F_\varphi := \text{localRefinement}(F_H, Q_D^H, \varphi);$
15. **for** each $w \in F_\varphi(w_0)$ **do**
16. $\tau :=$ the k -th highest aggregated score in $\Psi;$
17. **if** $\hat{s}(u, w) < \tau$ **then** Remove (w) from $F_\varphi(w_0);$ **continue**;
18. **for** each v such that $v \in F_\varphi(y_0)$ and $(v, w) \in F_\varphi(y_0, w_0)$ **do**
19. $s(u, v, w) := \text{updateScore}(u, v, w, \mathcal{M}_{\text{dis}}, \mathcal{M}_{\text{agg}}, \lambda);$
20. **if** $s(u, v, w) > \text{best}_w$ **then**
21. $\text{best}_w := s(u, v, w);$
22. **if** $\text{best}_w > \tau$ **then** Update Ψ and τ by $w;$
23. Compute (tighter) $\hat{y}(v)$ and $\Delta_\varphi^{\max}$ based on $F_\varphi;$
24. Update $\text{UB}_\varphi(u, v);$
25. Update $\hat{s}(u, v, w')$ for each $(v, w') \in F_\varphi(y_0, w_0);$
26. Update $\hat{s}(u, w)$ for each $w \in F_\varphi(w_0);$
27. $F_H := \text{groupRefinement}(\{F_\varphi | \varphi \in H\});$
28. $K_d(u) := \Psi;$
29. **Return** $K(u)$ and $K_d(u);$

Figure 12: Full version of TopK algorithm.

ing the local refinement results of the rules in H , including the local refinement induced by φ . In particular, when restricted to the variables of Q_φ , each candidate vertex that survives in C_H^{i+1} must survive the constraints of φ applied to the previous map C_H^i . Thus, for variables y_0 and w_0 , we have the following:

$$C_H^{i+1}(y_0) \subseteq F_\varphi(C_H^i(y_0)), \text{ and } C_H^{i+1}(w_0) \subseteq F_\varphi(C_H^i(w_0)).$$

Combining with the inclusions of the sequential algorithm, it yields

$$C_H^{i+1}(y_0) \subseteq C_{\varphi, \text{Seq}}^{i+1}(y_0), \text{ and } C_H^{i+1}(w_0) \subseteq C_{\varphi, \text{Seq}}^{i+1}(w_0).$$

The induction completes.

Since φ is chosen arbitrarily from $H \setminus \{\varphi_T\}$, taking the union over all OMRs yields $\mathcal{N}_{\text{Cpl}} \subseteq \mathcal{N}_{\text{Seq}}$. Consequently, $|\mathcal{N}_{\text{Cpl}}| \leq |\mathcal{N}_{\text{Seq}}|$. $\square$

Appendix C: Details of TopK Algorithm

Appendix C-1: Proof of Lemma 5

Proof: We proved this by distinguishing three cases. (a) If $\mathcal{M}_{\text{rec}}(u, v_1) > \mathcal{M}_{\text{rec}}(u, v_2) > \delta_H$, then v_1 is ranked before v_2 . (b) If $\mathcal{M}_{\text{rec}}(u, v_1) > \delta_H > \mathcal{M}_{\text{rec}}(u, v_2)$, then v_1 is selected into $I(u)$ and keeps its origin $\mathcal{M}_{\text{rec}}(u, v_1)$; for v_2 , its value is increased to δ_H , which is smaller than $\mathcal{M}_{\text{rec}}(u, v_1)$, and thus, v_1 is ranked before v_2 . (c) If $\delta_H > \mathcal{M}_{\text{rec}}(u, v_1) > \mathcal{M}_{\text{rec}}(u, v_2)$, both strengths are adjusted to δ_H and remains the same relative order. $\square$

Appendix C-2: Rule-level Upper Bound. Below we present an initialization of $\Delta_\varphi^{\max}$ for φ in the offline phase.

$$\Delta_\varphi^{\max} = \sum_{z \in \bar{x} \cup \bar{y}} \hat{s}(z),$$

where $\hat{s}(z)$ is the score upper bound of all vertices in G that can be mapped to a variable $z \in \bar{x} \cup \bar{y}$, i.e., $\hat{s}(z) = \max_{v \in h(z)} s(v)$.

Intuitively, instead of computing the exact score of $h(z)$ for each variable z , we initialize $\hat{s}(z)$ by the predicates in X since a $h(z)$ must satisfy the predicates constraint on z in X .

Initially, $\hat{y}(v)$ is initialized as 1. After one local refinement at φ , for every candidate pair $(v, w) \in F_\varphi(y_0, w_0)$, $\hat{y}(v)$ could be updated if some candidates w are discarded, and $\Delta_\varphi^{\max}$ is also updated since some candidates $C^H(z)$ on other variable z in φ may be removed. Then, the value of rule-level upper bound $\text{UB}_\varphi(u, v)$ can be tightened accordingly.

Appendix C-3: A Full Version of TopK Algorithm. Figure 12 shows a more detailed version of the TopK algorithm. It returns a top- k anchor set $K(u)$ and a top- k alternative set $K_d(u)$.

TopK first initializes an empty top- k heap Ψ and sets the anchor set $K(u)$ and alternative set $K_d(u)$ to be empty (Line 1). It then invokes TopKAnchor to obtain the top- k anchors $K(u)$ (Line 2). Based on $K(u)$, it constructs a rule group H of compatible OMRs from Σ_{OMR} via procedure constructGroup (Line 3). Unlike CTR, TopK restricts H to rules whose constraints on the anchor variable y_0 are satisfied by at least one $v \in K(u)$. It initializes a shared candidate map C^H and sets the group operator $F_H \leftarrow C^H$ (Line 4). Next, for each candidate alternative w , it initializes its aggregated score best_w and global upper bound $\hat{s}(u, w)$ (Line 5). For each $v \in F_H(y_0)$, it initializes the rule-level upper bound $\text{UB}_\varphi(u, v)$ for each $\varphi \in H$ (Lines 6–8). Then, for each candidate pair $(v, w) \in F_H(y_0, w_0)$, it initializes the candidate-level bound $\hat{s}(u, v, w)$ by the above definition, and updates the global upper bound $\hat{s}(u, w)$ (Lines 9–11).

The algorithm then performs coupled refinement until convergence (Lines 12–27). For each OMR $\varphi \in H$, it applies procedure localRefinement to enforce φ 's preconditions on F_H using Q_D^H (Line 14). It then enumerates each candidate alternative w in current $F_\varphi(w_0)$ (Lines 15–22). For each candidate alternative $w \in F_\varphi(w_0)$, it compares the bound $\hat{s}(u, w)$ with τ : If $\hat{s}(u, w) < \tau$, w is pruned from $F_\varphi(w_0)$ (Lines 16–17). Otherwise, it enumerates all pairs $(v, w) \in F_\varphi(y_0, w_0)$ tied to w , updating $s(u, v, w)$ via updateScore; if $s(u, v, w) > \text{best}_w$, it updates best_w ; if the updated aggregated score $\text{best}_w > \tau$, it updates Ψ and threshold τ with w (Lines 18–22). After processing all candidate alternatives, it recomputes the tighter bounds $\hat{y}(v)$ and $\Delta_\varphi^{\max}$ based on F_φ , since candidates of other variables in φ may have been removed after refinement (Line 23). Next, it updates the rule-level bounds $\text{UB}_\varphi(u, v)$, the candidate-level upper bounds $\hat{s}(u, v, w')$ for each $(v, w') \in F_\varphi(y_0, w_0)$, and the global upper bounds $\hat{s}(u, w')$ for each $(w') \in F_\varphi(w_0)$ (Lines 24–26). After processing all rules in H , the algorithm applies groupRefinement to update F_H (Line 27).

Once convergence is reached, it returns $K(u)$ and the selected alternatives in Ψ as $K_d(u)$ (Lines 28–29).

Appendix C-4: Proof of Theorem 6

Proof: We prove the theorem in two steps.

(1) We first show that $\hat{s}(u, w)$ is an upper bound of $s(u, w|K(u))$. For a fixed anchor $v \in K(u)$ and any alternative w , by definition, the score $s(u, v, w)$ is determined by the contributions of the OMRs

User profile				Favorite Anime				Latest Watched Anime			
User Name	Gender	Birthday	Age Group	Anime Name	Year	Genre	Short Description	Anime Name	Year	Genre	Short Description
Visceras	Male	Aug 18, 1997	Post-95s	Clannad After Story	2008	Drama/ romance/ slice of life/ supernatural	A sequel to Clannad that focuses on adulthood, family, loss, and emotional growth.	Terror in Resonance	2014	Psychological thriller/ mystery/ crime	Two teenage terrorists challenge society through a series of attacks and riddles.
				Fate/ Zero	2011	Action/ dark fantasy/ supernatural	A battle-royale story in which mages and heroic spirits fight in the Holy Grail War.	Phantom: Requiem for the Phantom	2009	Psychological/ crime/ action/ drama	A young man is drawn into the underworld after being turned into an assassin.
				Death Note	2006	Psychological thriller/ mystery/ supernatural	A high school student gains a notebook that can kill anyone whose name is written in it.	Tokyo Ghoul	2014	Dark fantasy/ horror/ action/ supernatural	A college student becomes half-ghoul and struggles between human and monster worlds.
				Wolf Children	2012	Drama/ fantasy/ family/ slice of life	A heartfelt film about a mother raising two half-wolf children while protecting their secret.	Fate/ Zero	2011	Action/ dark fantasy/ supernatural	A battle-royale story in which mages and heroic spirits fight in the Holy Grail War.

Table 3: The profile and historical preferences of user (ID:“user:Visceras”) on Anime.

Traditional Recommendation (Anchor)				Diversified Recommendation (Alternative)				
Anime Name	Year	Genre	Short Description	Methods	Anime Name	Year	Genre	Short Description
Death Note	2006	Psychological thriller/ mystery/ supernatural	A high school student gains a notebook that can kill anyone whose name is written in it.	RAPID	Howl’s Moving Castle	2004	Fantasy/ adventure/ romance	A young woman cursed into an old body becomes involved with a mysterious wizard and his moving castle.
				Random	Blue Spring Ride	2014	Romance/ drama/ slice of life/ school	A high school girl reconnects with her first love and navigates youth, friendship, and emotional change.
				SameTopic	Death Note: Rewrite	2007	Psychological thriller/ mystery/ supernatural	A recap film that reorganizes the main events of Death Note into a condensed retelling.
				OpenMind	The Future Diary	2011	Psychological thriller/survival/ supernatural/ action	A boy is forced into a deadly game where participants use diaries that can predict the future

Table 4: Traditional recommendation and the diversified recommendations by evaluated methods.

in H together with the accuracy term $s_{acc}(u, w)$. For each $\varphi \in H$, $UB_{\varphi}(u, v)$ is defined as the maximum possible contribution that φ can make to any candidate alternative at (u, v) . Hence, for each w , $\Delta_{\varphi}(u, v, w) \leq UB_{\varphi}(u, v)$. Summing over all OMRs in H , we have $s(u, v, w) \leq \hat{s}(u, v, w)$. Since this holds for each $v \in K(u)$, taking the maximum over all selected anchors gives the following inequalities: $s(u, v|K(u)) = \max_{v \in K(u)} s(u, v, w) \leq \max_{v \in K(u)} \hat{s}(u, v, w) = \hat{s}(u, w)$. Thus, $\hat{s}(u, w)$ is a valid upper bound on the aggregated score of w .

(2) We show that pruning an alternative w when $\hat{s}(u, w) \leq \tau$ is safe. Suppose that at some iteration, for a candidate alternative w , we have $\hat{s}(u, w) < \tau$. By step (1), $\hat{s}(u, w)$ is an upper bound on the true aggregated score of w , i.e.,

$$s(u, w | K(u)) \leq \hat{s}(u, w) < \tau.$$

Hence the true score of w is already strictly below the current top- k threshold, and thus w cannot belong to the current top- k result.

It remains to show that w cannot become feasible later. During refinement, candidates are only removed from the match sets. Thus, all rule-level upper bounds $UB_{\varphi}(u, v)$, pairwise bounds $\hat{s}(u, v, w)$, and aggregated bounds $\hat{s}(u, w)$ are monotonically non-increasing. In contrast, the heap threshold τ is monotonically non-decreasing, since the top- k heap only replaces lower-scoring entries with higher-scoring ones. Consequently, once $\hat{s}(u, w) < \tau$ holds at some iteration, it continues to hold in all subsequent iterations. Thus, w can never re-enter the top- k answer, and pruning w is safe. $\square$

Appendix D: Details of Experiments

Appendix D-1: Evaluation metrics. Filter Bubble Penetration (FBP) evaluates the proportion of recommended items outside users’ echo chamber, defined as:

$$FBP = \frac{1}{n} \sum_u \frac{|Out(\mathcal{R}_u)|}{|\mathcal{R}_u|},$$

where $Out(\mathcal{R}_u)$ is the set of items outside u ’s echo chamber, i.e., items whose average similarity score to the items u historically interacted with is below threshold β . This metric follows the exploration metric in [25] that measures the extent to which the

recommended items deviate from a user’s known interests. The value of threshold β is determined by the average similarity score of each pair of items in the user u ’s historical interactions, yielding 0.72, 0.63 and 0.66 on Anime, Steam and Book, respectively.

Appendix D-2: User study. We conducted a user study to assess whether OpenMind is better received than existing methods in recommending alternatives with diverse viewpoints. We compared OpenMind with RAPID, Random and SameTopic. This study was conducted on the user-anime review dataset Anime. We provide both qualitative and quantitative evaluations.

Qualitative evaluation. For a user (ID: “user:Visceras”) on Anime, Table 3 shows his profile, favorite anime, and most recently watched anime. Specifically, he is a young male born after 1995. Based on his watched-anime list (i.e., the positive interactions used to train the recommender system), this user appears to prefer anime characterized by *dark tone*, *psychological tension*, *supernatural elements*, and *action-oriented narratives*. Based on this profile and interaction history, a traditional recommender system suggests an anime *Death Note* to this user, which is highly consistent with his historical preferences, as shown in Table 4. Building upon this traditional recommendation, each evaluated method further provides one diversified recommendation for diversity, also reported in Table 4.

Specifically, RAPID recommends an adventure anime *Howl’s Moving Castle*, in which a young woman who is cursed into an old body becomes involved with a mysterious wizard and his moving castle. However, this anime differs substantially from *Death Note* in genre, and is also misaligned with the user’s preferences; therefore, it is unlikely to be accepted by the user.

Random randomly samples alternatives from the entire anime dataset and recommends *Blue Spring Ride*. However, this recommendation also does not fall within the user’s preferences. SameTopic randomly picks an anime with a genre similar to *Death Note*, recommending *Death Note: Rewrite*. Nevertheless, this anime is essentially a recap of the anime already recommended by the traditional rec-

ommender, and thus remains within the same perspective.

In contrast, our proposed OpenMind recommends an anime *The Future Diary*, which not only shares similar genres with *Death Note* but also aligns well with the user’s preferences. More importantly, it provides a different narrative style, thus broadening the user’s horizons and helping to break filter bubbles.

Quantitative evaluation. We adopted four metrics: (1) *Acceptability* (Acc), measuring whether the user would likely accept the recommended items, (2) *Topic Consistency* (TopCon), measuring whether the recommended items remain within the same topic (category) as the anchor, (3) *View Divergence* (ViewD), evaluating whether the recommended items present different viewpoints/perspectives, and (4) *Overall* (All) that considers all the above metrics simultaneously.

We further applied cross-check questions to filter invalid responses. More specifically, we presented two questions for each metric, which asked the participants to select the best recommenda-

tion and best two recommendations *w.r.t.* each metric, respectively. If the best-two recommendations do not include the best one, the response was marked as invalid. In total, we collected 76 valid responses, out of 84 responses. For each metric, each respondent selected the best two recommendations.

As shown in Figure 10(l), 61 of the responses indicate that OpenMind performs the best overall. Although RAPID and Random have higher ViewD than OpenMind as both provide across-topic items, they sacrifice the overall performance. Compared with RAPID (resp. Random), OpenMind achieves 39.48% (resp. 55.27%), 67.1% (resp. 75%) and 17.1% (resp. 60.52%) higher Acc, TopCon and All, respectively, verifying the effectiveness of balancing both acceptability and diversity in alternative generation. Compared with SameTopic, OpenMind improves ViewD by 27.63%, showing that the topic preservation alone is insufficient for exposing users to opposing views.